Capítulo 17

Calidad de servicio

Al terminar este capítulo, entenderás:

- Los parámetros básicos de Calidad de Servicio: tasa de transferencia, latencia, *jitter*, pérdida de paquetes.
- Cómo medir tasa de transferencia, latencia y *jitter*.
- Cómo el control de flujo TCP actúa como limitador de tasa.
- Qué es el marcado y clasificación de paquetes.
- Cómo operan policing y shaping.
- Cómo funciona la Calidad de Servicio en Linux.

Se dice de IP que es un protocolo de «mejor esfuerzo» *(best effort)*, que no es más que una forma bonita de decir que hace lo que buenamente puede y eso, en función de las condiciones de la red, podría ser absolutamente nada. La integridad, el orden o incluso la propia entrega son prestaciones que IP consigue eventualmente, pero no están en absoluto aseguradas. Aunque TCP sí ofrece estas garantías, no todas las aplicaciones funcionan bien con TCP, e incluso cuando es así, hay otras prestaciones deseables que no ofrece. Por ejemplo, la tecnología TCP/IP básica no puede garantizar (ni siquiera lo intenta) una tasa de transferencia mínima, o una latencia o *jitter* por debajo de ciertos umbrales.

Una de las limitaciones clave de la tecnología de conmutación de paquetes, en la que se basa IP e Internet, es que no dispone de *control de admisión*, es decir, nunca niega una nueva solicitud de envío o conexión por mucho que la interred ya esté sufriendo una carga excesiva o incluso congestión.

El control de admisión es la característica típica de las redes de conmutación de circuitos, como la de telefonía. La red telefónica puede cuantificar la cantidad de recursos disponibles y rechazar el servicio solicitado por un cliente si estima que no lo va a poder satisfacer adecuadamente: el clásico aviso de «todas las líneas están ocupadas». La conveniencia del control de admisión es fácil de entender, pero existen otras formas, no tan obvias, de garantizar el servicio.

La QoS (Quality of Service) engloba los mecanismos que tratan de garantizar determinado nivel en los parámetros de tráfico para los servicios que ofrece una interred de conmutación de paquetes. La idea que subyace en la QoS es que la red debería asignar sus recursos en función de las necesidades de cada usuario, tipo de flujo o servicio, bajo la premisa de que no todos necesitan las mismas prestaciones. Por ejemplo, una llamada de voz requiere baja latencia, pero puede aceptar cierta pérdida de mensajes, mientras que la descarga de un archivo puede tolerar alta latencia, pero ninguna pérdida de datos. Fíjate que QoS no implica cotas muy exigentes de rendimiento, solo garantizar ciertos mínimos, que no tienen por qué ser altos, pero que deben cumplirse.

Un flujo de red (en los términos que se definió en § 6.3) es la unidad de tráfico que se trata como un todo a la hora de clasificar, priorizar o asignar recursos, y es el elemento básico sobre el que trabja el pipeline de QoS.

Los parámetros QoS básicos de un flujo se han comentado ya: tasa de transferencia, latencia, *jitter* y tasa de pérdida de paquetes —o confiabilidad en general. Es importante entender que QoS es un juego de suma cero: los recursos son limitados (y normalmente conocidos) y los que consume un flujo no están disponibles para los demás.

Un ejemplo con la tasa de transferencia, que es un parámetro fácil de entender: imagina un enlace que, por construcción, está limitado a 100 Mbps: QoS debe garantizar que la suma de las tasas utilizadas por todos los flujos no exceda ese límite. Su papel, por tanto, es doble: asignar a cada flujo los recursos que necesita —aquí, ancho de banda— y evitar que, en conjunto, excedan la capacidad disponible. Si ese tipo de control no existe, cuando el enlace se satura, se descartan paquetes arbitrarios con consecuencias impredecibles; con él, es posible elegir qué paquetes conviene sacrificar para minimizar el daño.

A veces, el límite no es la capacidad física del canal, sino un valor acordado, parte de un contrato comercial entre un proveedor y un cliente, en concreto el ancho de banda contratado se denomina CIR (Committed Information Rate). Imagina un ISP que ofrece a sus clientes conexión de fibra con 1 Gbps de descarga (ese es el CIR). Por encima de eso, el proveedor puede ofrecer un extra si tiene capacidad libre, pero sin garantías. Esa capacidad extra es el EIR (Excess Information Rate)[1].

El contrato no es solo retórica. Literalmente existe un contrato que se conoce como SLA (Service Level Agreement) o en español Acuerdo de Nivel de Servicio, y en los casos más exigentes, aparte de la tasa, incluye objeti-

[1] También se suele hablar de PIR (Peak Information Rate), que es CIR + EIR

vos cuantificables (SLO) y métricas para validar su cumplimiento (SLI). Un ejemplo del tipo de información que incluye un SLA:

- Tasa de transferencia garantizada: 100 Mbps.
- Tasa de transferencia máxima: 200 Mbps.
- Latencia máxima: 20 ms.
- *jitter* máximo: 5 ms.
- Tasa máxima de pérdida de paquetes: 0,1 %.
- Disponibilidad: 99,95 %.

Este capítulo estudia estos parámetros, su importancia, causas y consecuencias, y la forma de medirlos; qué es y cómo funciona un pipeline QoS, qué componentes lo forman y qué puede ofrecer; y cómo Linux implementa QoS y cómo se configura.

17.1. Latencia y *jitter*

La latencia se ha mencionado ya de forma informal en capítulos anteriores; conviene ahora dar una definición concreta y, de paso, la de algunos conceptos relacionados.

El **retardo** *(delay)*, erróneamente utilizado como sinónimo de latencia, se refiere al tiempo que le lleva al mensaje atravesar un elemento específico. Se consideran varios tipos de retardo: de procesamiento, de transmisión, de cola, de propagación, etc.

La **latencia** es el tiempo que le lleva a un mensaje viajar desde el emisor hasta el destino, es decir, es la suma de los retardos de todos los elementos que componen el camino.

El **tiempo de respuesta** *(response time)* es el tiempo total que transcurre desde que la solicitud sale del emisor hasta que la respuesta le llega de vuelta. Incluye el retardo de la red en el camino de ida y vuelta, el tiempo de procesamiento del servidor y el que requiere el SO de cada nodo para procesar ambos mensajes.

El **tiempo de ida y vuelta** (RTT) es el tiempo que tarda un mensaje en ir del emisor al destino y regresar al emisor. A diferencia del tiempo de respuesta, en este solo se contabiliza el tiempo de transmisión.

El ***jitter*** es la variación del retardo o la latencia, específicamente en relación a los otros mensajes del mismo flujo.

17.1.1. Causas del retardo y del *jitter*

De los tipos de retardo, el más relacionado con la red es el de propagación. Es el tiempo que le lleva a un mensaje viajar a través del medio físico

en el enlace. Se debe a dos factores principales: la distancia física entre los extremos y la velocidad de propagación de la señal en ese medio de transmisión. Se calcula como d/v siendo d la longitud del enlace y v la velocidad de propagación.

El retardo de transmisión es el tiempo que lleva colocar los bits del mensaje en el canal por medio de la interfaz de red. Se calcula como L/R siendo L el tamaño del mensaje (en bits) y R la velocidad de transmisión.

Por ejemplo, la latencia para enviar un mensaje por un canal de fibra óptica de 50 km, con velocidad de propagación de $2 \cdot 10^8$ m/s y un mensaje de 1 kb a través de un enlace de 1 Gbps, sería:

$$\text{latencia} = \text{retardo de transmisión} + \text{retardo de propagación}$$

$$\text{latencia} = \frac{1 \cdot 10^3}{1 \cdot 10^9} + \frac{50 \cdot 10^3}{2 \cdot 10^8} = 1\ \mu\text{s} + 0{,}25\ \text{ms} \approx 251\ \mu\text{s}$$

Este cálculo desprecia el retardo de procesamiento y de cola que ocurre en el emisor para preparar el mensaje y pedirle al SO que lo envíe a través de un socket, así como la gestión del SO con la NIC. También asume que no hay dispositivos intermedios como routers o switches. El cálculo de la latencia real (extremo a extremo) depende de otros elementos y factores en una situación real.

Ambos retardos de transmisión y propagación son bastante deterministas y dependen únicamente de valores conocidos y fijos. El retardo de procesamiento por otra parte puede variar dependiendo de la carga del nodo e incluso de los propios datos si requiere por ejemplo tareas de codificación o cifrado.

Por último, el retardo de cola *(queuing delay)* es el tiempo que el mensaje tiene que esperar en las colas de salida y entrada de los nodos origen y destino, y de dispositivos de interconexión. Es el más variable porque depende de decisiones de los algoritmos involucrados aparte de por la propia carga de la red. La variación del retardo de cola es la causa principal del *jitter*. Si en un flujo, uno de los paquetes se ve obligado a esperar en la cola de un router durante 10ms y el siguiente solo tiene que esperar 2ms, está introduciendo una variación (*jitter*) de 8ms.

El *jitter* es especialmente perjudicial para las aplicaciones que reproducen un flujo en tiempo real, como la telefonía IP, el streaming multimedia o los videojuegos en línea, porque necesitan entregar los datos a un ritmo constante. Para absorber esa variación, estas aplicaciones intercalan un *jitter buffer* entre la recepción y la reproducción: retienen brevemente los

mensajes recibidos y los entregan a la aplicación al ritmo de consumo del reproductor. Cuanto mayor es el *jitter*, mayor debe ser el buffer para así evitar cortes en la reproducción, aunque eso implica retardo adicional. Por esta razón, el *jitter* resulta may problemático para el tráfico en tiempo real mientras que es prácticamente inocuo para una transferencia masiva de datos *(bulk data)*, donde los mensajes se pueden almacenar y reordenar sin ninguna urgencia.

17.1.2. Medición del retardo

Los distintos tipos de retardo se pueden calcular en términos teóricos, pero es complicado medirlos en la práctica porque implicaría tener puntos de control sincronizados en todos los componentes involucrados. También la latencia es difícil de calcular ya que es necesario que emisor y receptor cuenten con relojes locales sincronizados con suficiente precisión. Los protocolos de sincronización como NTP tienen una precisión limitada precisamente por esa misma latencia. Para lograr ese tipo de sincronización se requiere una fuente externa de sincronización como GPS.

Por todo esto, es mucho más sencillo y práctico medir tiempo de ida y vuelta, como hace `ping`, y estimar la latencia como la mitad de ese valor, aunque teniendo muy claro que no es exacto: la latencia de uno de los sentidos podría ser mayor que la del otro, y no hay forma de saberlo.

```
$ ping example.net
PING example.net (172.66.175.59) 56(84) bytes of data.
64 bytes from 172.66.175.59: icmp_seq=1 ttl=57 time=8.24 ms
64 bytes from 172.66.175.59: icmp_seq=2 ttl=57 time=8.90 ms
64 bytes from 172.66.175.59: icmp_seq=3 ttl=57 time=10.1 ms
64 bytes from 172.66.175.59: icmp_seq=4 ttl=57 time=7.21 ms
^C
--- example.net ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 7.206/8.606/10.083/1.044 ms
```

LISTADO 17.1: Medición de RTT con `ping`.

17.1.3. Cálculo del *jitter*

Si estimamos la latencia como RTT/2, deberemos estimar el *jitter* como la mitad de la variación del RTT. La forma más simple para obtener el *jitter* es calcular las diferencias entre medidas consecutivas de RTT y calcular después una media.

$$J = \frac{1}{2}\frac{1}{N-1}\sum_{i=2}^{N}|RTT_i - RTT_{i-1}| \tag{17.1}$$

donde:

J es el *jitter*, RTT_i es la medida de RTT del paquete i, y N es el número total de paquetes.

Aplicada a los datos de la captura 17.1, el *jitter* que obtenemos es:

$$\begin{aligned} J &= \frac{1}{2}\frac{1}{4-1}\left(|8{,}90 - 8{,}24| + |10{,}1 - 8{,}90| + |7{,}21 - 10{,}1|\right) \\ &= \frac{1}{6}\left(0{,}66 + 1{,}20 + 2{,}89\right) = \frac{1}{6}\cdot 4{,}75 \\ &\approx 0{,}792 \text{ ms} \end{aligned}$$

Sin embargo, el *jitter* se suele calcular a menudo como una EMA denominada *interarrival jitter*. Es la fórmula que aplica RTP tal como está definida en la RFC 1889 [53]. De ese modo, el cálculo es más estable y resiste mejor el ruido (valores atípicos puntuales). La fórmula es:

$$J_i = \frac{15}{16}J_{i-1} + \frac{1}{16}|L_i - L_{i-1}| \tag{17.2}$$

donde:

L es la latencia, J_{i-1} es el *jitter* previamente calculado y 1/16 es el factor de suavizado.

ECUACIÓN 17.1: Cálculo del *jitter* como media móvil exponencial.

Sin embargo, no es eso lo que hace `ping`. El valor `mdev` que muestra en sus estadísticas (Listado 17.1) no es el *jitter* sino la desviación estándar de los RTT, que también da una buena idea de la dispersión de la latencia.

$$\text{mdev} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(RTT_i - \overline{RTT}\right)^2} \tag{17.3}$$

donde:

$\overline{RTT}$ es el RTT promedio y N es el número de muestras.

Aplicando esta fórmula a los datos de la captura 17.1 se obtiene:

$$\begin{aligned}\text{mdev} &= \sqrt{\frac{(8{,}24-8{,}6125)^2+(8{,}90-8{,}6125)^2+(10{,}1-8{,}6125)^2+(7{,}21-8{,}6125)^2}{4}}\\ &= \sqrt{\frac{0{,}1388+0{,}0827+2{,}2127+1{,}9670}{4}}\\ &\approx 1{,}049 \text{ ms}\end{aligned}$$

La pequeña diferencia con el valor que aparece en la salida de `ping` (1,044) se debe al redondeo que realiza el programa al mostrar los datos.

Con `mtr` también puedes medir RTT y desviación típica, con la diferencia de que este realiza la medida para todos los routers intermedios.

```
~$ mtr --report-wide --report-cycles 4 sidney.example.net
Start: 2026-07-15T16:31:23+0200
HOST: amy                Loss%  Snt  Last   Avg  Best  Wrst StDev
  1.|-- _gateway          0.0%    4   2.5   6.0   2.5  14.5   5.7
  2.|-- 192.168.0.1       0.0%    4  10.5   5.6   3.5  10.5   3.3
  3.|-- ???              100.0    4   0.0   0.0   0.0   0.0   0.0
  4.|-- 188.114.108.49   75.0%    4   8.2   8.2   8.2   8.2   0.0
  5.|-- 188.114.108.48    0.0%    4  16.5  10.4   8.2  16.5   4.0
  6.|-- 188.114.108.1     0.0%    4  20.1  12.2   7.9  20.1   5.5
  7.|-- 188.114.108.7     0.0%    4  10.0  31.9  10.0  58.4  25.3
  8.|-- 172.66.175.59     0.0%    4   8.5   9.1   7.4  12.9   2.6
```

El programa `iperf3` sí ofrece una medición de *jitter* canónica aplicando la Fórmula 17.1. Aunque este es un programa cliente-servidor puede medir el *jitter* sin necesidad de sincronización precisa de los relojes. Como el cálculo depende de diferencias entre medidas, la discrepancia entre relojes local y remoto *(drift)* no afecta al resultado.

```
~$ iperf3 --time 5 --bitrate 500M --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 54954 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate    otal Datagrams
[  5]   0.00-1.00   7.30 MBytes  61.2 Mbps  5314
[  5]   1.00-2.00   6.90 MBytes  57.9 Mbps  5028
[  5]   2.00-3.00   7.15 MBytes  59.9 Mbps  5203
[  5]   3.00-4.00   7.07 MBytes  59.3 Mbps  5148
[  5]   4.00-5.00   6.29 MBytes  52.7 Mbps  4578
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   34.7 MBytes  58.2 Mbps  0.000 ms  0/25271 (0%)  sender
[  5]   0.00-5.09   34.7 MBytes  57.2 Mbps  0.141 ms  0/25269 (0%)  receiver
```

17.2. Cálculo de la tasa de transferencia

La tasa de transferencia mide la velocidad con la que se mueven los datos, ya sea a través de una red, un enlace, un flujo UDP, una conexión TCP, etc.

La tasa se mide en bits por segundo (b/s o bps), bytes por segundo (B/s) y sus múltiplos (ver § D.3).

Se distingue entre dos tipos de tasa:

Throughput
: o tasa bruta, es la cantidad total de datos transmitidos en un período de tiempo. Además de la carga útil, incluye cabeceras, reconocimientos, retransmisiones o cualquier otro tipo de información de control; todo eso que llamamos sobrecarga de protocolos.

Goodput
: o tasa neta, considera solo los datos efectivos, la carga útil procedente del usuario o la aplicación.

En realidad, ambas tasas se pueden medir en distintas capas, considerando diferentes cabeceras y sus mecanismos de control. La tasa bruta de más bajo nivel debería contabilizar hasta las cabeceras de enlace, por lo que tiene que medirse directamente desde la interfaz de red.

La tasa neta se suele medir sobre el nivel de transporte, es decir, con los datos que se envían o reciben con las primitivas del socket (como veremos en los ejemplos de código) o sobre el nivel de aplicación, que sería el caso en el que únicamente se cuentan los datos relevantes para el usuario, como por ejemplo, un fichero descargado desde un servidor web con HTTP.

También es útil distinguir la tasa en los extremos y en la red:

Tasa de envío (T_s)
: es la velocidad con la que el proceso emisor entrega datos al SO a través de un socket, lo que, en el caso de TCP, no implica necesariamente que esos datos estén saliendo a la red y puede que simplemente se estén almacenando en el buffer de envío.

Tasa de recepción[2] **(T_r)**
: es la velocidad con la que el proceso receptor recoge datos a través de un socket, que con TCP implica leer datos del buffer de recepción.

Tasa de red[3] **(T_n)**
: es la velocidad a la que los datos se mueven a través de la red. Esta tasa no se puede medir directamente en los extremos, pero se puede estimar a partir de las tasas de envío y recepción.

[2] en inglés: *drain rate*
[3] en inglés: *network rate*

Lógicamente para cualquiera de las tres, pueden ser útiles tanto la tasa bruta como la neta. Los datos se envían en bloques (segmentos o datagramas) que pueden tener tamaños distintos y que son enviados/recibidos a intervalos posiblemente irregulares. Eso condiciona el cálculo y da lugar a los métodos más habituales, descritos a continuación.

17.2.1. Tasa instantánea

Es el cálculo más sencillo, y probablemente también el menos útil. Consiste en contabilizar únicamente los datos transmitidos desde la medición anterior considerando el tiempo transcurrido desde entonces. Formalmente:

$$R_{\text{inst}} = \frac{\Delta B}{\Delta t} \tag{17.4}$$

donde:

> R_{inst} es la tasa *(rate)* instantánea (bytes por segundo), ΔB es la cantidad de datos transmitida en el último bloque (bytes), y Δt es el tiempo transcurrido desde el envío del bloque anterior (segundos).

La tasa instantánea puede variar drásticamente de un mensaje al siguiente. Se dice que es muy «sensible al ruido», y por eso raramente resulta útil.

En Python la tasa de envío neta instantánea podría calcularse como en el siguiente fragmento[4]. Fíjate que usamos `time.monotonic()` en lugar de `time.time()` para que el cálculo de los delta no se vea afectado por ajustes en el reloj del sistema.

```
while 1:
    last_time = time.monotonic()
    data = get_data()
    sock.sendall(data)
    byte_rate = len(data) / (time.monotonic() - last_time)
    print(byte_rate)
```

17.2.2. Promedio acumulado (CA)

En el otro extremo de la tasa instantánea está el promedio total acumulado o CA (Cumulative Average). Consiste simplemente en calcular la media de toda la transmisión hasta el momento, es decir, el total de datos transmitidos partido por el tiempo transcurrido desde el inicio.

$$R_{\text{avg}}(t) = \frac{B(t) - B(t_0)}{t - t_0} \tag{17.5}$$

[4] Adaptarlo a la tasa de recepción es directo

donde:

> $R_{\text{avg}}(t)$ es la tasa media acumulada (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t_0)$ es la cantidad total de datos transmitida en el instante t_0, y $t - t_0$ es el tiempo transcurrido desde el comienzo (segundos).

Lógicamente sufre del efecto contrario a la tasa instantánea. No se adapta en absoluto a las variaciones (nada reactivo), por lo que una reducción o aumento momentáneo de la tasa queda completamente enmascarado.

En Python:

```
start_time = time.monotonic()
sent = 0
while 1:
    data = get_data()
    sock.sendall(data)
    sent += len(data)
    elapsed = time.monotonic() - start_time
    byte_rate = sent / elapsed
    print(byte_rate)
```

LISTADO 17.2: Promedio acumulado

17.2.3. Media móvil simple (SMA)

Una técnica intermedia que ofrece información actualizada, pero más estable que la tasa instantánea, es la 'media móvil simple' o SMA (Simple Moving Average). Se basa en definir una ventana de tiempo que abarca solo los w segundos previos considerando únicamente los datos transmitidos en ese lapso.

$$R_{\text{SMA}}(t) = \frac{B(t) - B(t-w)}{w} \tag{17.6}$$

donde:

> $R_{\text{SMA}}(t)$ es la tasa media móvil simple (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t-w)$ es la cantidad total de datos transmitida hace w segundos, y w es el tamaño de la ventana (segundos).

Aunque es una medida mucho más reactiva que el promedio acumulado, los nuevos datos tardan algún tiempo ($w/2$ segundos) en reflejarse en la tasa.

En Python:

```
class SMA_RateMeter:
    def __init__(self, window_size=5):
        self.window_size = window_size
        self.window = collections.deque()
```

```
    def update(self, sent_bytes):
        now = time.monotonic()
        self.window.append((now, sent_bytes))

        # Eliminar datos fuera de la ventana
        while self.window and (now - self.window[0][0]) > self.window_size:
            self.window.popleft()

    @property
    def rate(self):
        if not self.window:
            return 0

        window_bytes = sum(b for t, b in self.window)
        elapsed = self.window[-1][0] - self.window[0][0]
        return window_bytes / elapsed if elapsed > 0 else 0

sma = SMA_RateMeter(window_size=5)
while 1:
    data = get_data()
    sock.sendall(data)
    sma.update(len(data))
    print(sma.rate)
```

17.2.4. Media móvil exponencial (EMA)

SMA también tiene un problema: los datos nuevos de la ventana tienen el mismo peso que los antiguos, por lo que la tasa no refleja bien las variaciones en el tráfico (sigue siendo poco reactivo). Para paliar ese efecto se puede usar una 'media móvil exponencial', o EMA (Exponential Moving Average), que aplica un decaimiento exponencial, asignando menos peso a los datos más antiguos.

Se calcula utilizando la siguiente fórmula:

$$R_{\text{EMA}}(t) = \alpha \cdot R_{\text{inst}}(t) + (1 - \alpha) \cdot R_{\text{EMA}}(t-1) \tag{17.7}$$

donde:

> $R_{\text{EMA}}(t)$ es la tasa media móvil exponencial, $R_{\text{inst}}(t)$ es la tasa instantánea en el instante t, y α es el factor de suavizado ($0 < \alpha < 1$).

El factor de suavizado α permite ajustar la reactividad del cálculo. Así un valor de 1 la hace equivalente a la tasa instantánea, y valores menores aumentan el peso de los cálculos anteriores. Un valor típico es $\alpha = 0{,}1$.

En Python, una implementación básica del EMA podría ser:

```
class EMA_RateMeter:
    def __init__(self, alpha=0.1):
        self.alpha = alpha
```

```
        self.rate = 0
        self.last = time.monotonic()

    def update(self, sent_bytes):
        elapsed = time.monotonic() - self.last
        self.rate = self.alpha * sent_bytes / elapsed + (1 - self.alpha) * self.rate
        self.last = time.monotonic()

ema = EMA_RateMeter()
while 1:
    data = get_data()
    sock.sendall(data)
    ema.update(len(data))
    print(ema.rate)
```

17.2.5. Consideraciones sobre el cálculo de tasa

Tampoco EMA es perfecto. Al comienzo de una conexión TCP, el emisor puede enviar —invocando `send()`— una gran cantidad de datos en muy poco tiempo porque en realidad esos datos se almacenan en el buffer de envío y recepción, pero no están siendo aún consumidos realmente por el proceso receptor.

Esto genera cálculos de tasa instantánea (en los que se basa EMA) enormes e irreales, porque a fin de cuentas lo que se está midiendo en esos primeros instantes es la tasa entre el proceso y la memoria local y remota, pero no entre los procesos emisor y receptor, que es lo que proporciona una medida representativa. Estos valores tan altos falsean el resultado, e incluso con el decaimiento exponencial, puede llevar mucho tiempo compensar ese efecto. En esta situación no mejora la reactividad, que acaba siendo tan pobre como la del promedio acumulado.

Hay algunas opciones que pueden ayudar a paliar este problema:

- Ignorar la tasa instantánea los primeros segundos de la conexión (período de calentamiento o *warmup*).
- Descartar lapsos (Δt) demasiado pequeños.
- Utilizar un α bajo para Δt pequeños. Se puede calcular como $\alpha = 1 - e^{-dt/\tau}$ siendo τ una constante que determina la reactividad.
- Reducir el tamaño de los buffers TCP de envío y recepción.
- No usar EMA en el emisor.
- Usar SMA en lugar de EMA durante el calentamiento o si no se necesita alta reactividad.

Aplicaremos algunas de estas técnicas a los ejemplos de este capítulo.

17.3. Control de flujo TCP como limitador de tasa

Sabemos que el control de flujo TCP permite al receptor controlar la cantidad de datos que el emisor le envía de acuerdo al espacio del que dispone (la ventana que anuncia). Aunque no es su objetivo principal, este mecanismo se puede utilizar para conseguir un control de la tasa de transferencia rudimentario.

Considerando solo uno de los sentidos de la comunicación, asumamos por un momento que el emisor siempre tiene datos disponibles y los puede enviar siempre que sea posible. Con esta premisa se pueden dar tres situaciones en función de la tasa de recepción (T_r) respecto a la velocidad de la red (T_n). Lógicamente la tasa de emisión (T_s) estará limitada por la menor de las otras dos.

- Si $T_r < T_n$, el buffer de recepción se llena, la ventana se cierra, el buffer de emisión también se llena y el emisor se bloquea *(stall)* en espera de que se libere espacio cuando la ventana vuelva a abrir. En esta situación, la tasa de emisión está limitada por el proceso receptor.
- Si $T_r > T_n$, el buffer de recepción queda vacío y el receptor quedará bloqueado en espera de nuevos datos. En esta situación, la tasa de emisión está limitada por la red.
- Si $T_r \approx T_n$, el buffer de recepción nunca se llena ni queda vacío. Ni el emisor ni el receptor se bloquean.

Lógicamente, los buffers de emisión y recepción existen precisamente porque el tercer caso, aunque deseable, es poco frecuente. Estas tres velocidades: emisión, red y recepción suelen ser dispares y cambiantes. Los buffers amortiguan esas diferencias y permiten un flujo más estable. Sin embargo, obviando otros controles —como el de congestión— cuando la ventana está abierta el emisor TCP envía datos a la máxima tasa posible en ese momento, y eso frecuentemente provoca bloqueos en ambos extremos, lo que influye en parte en que T_r y T_n varíen constantemente. Además el algoritmo de Nagle trata de evitar segmentos pequeños, lo que alarga esos bloqueos. La conclusión es que cuando existe disparidad en el desempeño de emisor y receptor, el mecanismo de control de flujo (y en menor medida el de congestión) provoca un flujo a ráfagas, es decir, momentos en los que se envían datos a la máxima velocidad posible y momentos en los que no se envía nada (porque la ventana está cerrada).

La consecuencia directa de todo esto es que si el receptor limita intencionadamente la tasa de recepción, la tasa media de emisión se verá también limitada, aunque su dispersión (*p. ej.* la desviación típica) será muy alta debido al flujo en ráfagas.

En principio una limitación de tasa de este tipo no es incorrecta, pero cuando ocurre de forma generalizada puede provocar efectos perjudiciales como el aumento de la latencia o el *jitter*. Lo veremos más adelante.

17.3.1. Limitación de tasa en el receptor

Se puede conseguir bloqueando intencionadamente el proceso con `time.sleep()` el tiempo necesario para aproximar la tasa medida a la tasa deseada. El siguiente código calcula la tasa con el método de promedio acumulado (`ca_rate`), aunque puede sustituirse por cualquier otro método de cálculo de tasa.

```
1  target_rate_kBps = 200 * 1000  # 200 kB/s
2  start_time = time.monotonic()
3  received = 0
4  while 1:
5      data = sock.recv(4096)
6      if not data:
7          break
8
9      received += len(data)
10     elapsed = time.monotonic() - start_time
11     ca_rate = received / elapsed
12
13     if ca_rate <= target_rate_kBps:
14         continue
15
16     adjust_time = (received / target_rate_kBps) - elapsed
17     if adjust_time > 0:
18         time.sleep(adjust_time)
```

LISTADO 17.3: Limitación de la tasa de recepción (medida con promedio acumulado)

Hasta la **línea 14** es equivalente al Listado 17.2 salvo que aquí estamos calculando la tasa de recepción. Si este cálculo está por debajo de la tasa objetivo (`target_rate_kBps`) no hace nada, pero si está por encima, realiza un ajuste aumentando el tiempo (el denominador de la Fórmula 17.5). El ajuste se obtiene como la diferencia entre el tiempo realmente transcurrido y el tiempo que corresponde a la tasa objetivo (**línea 16**).

El ejemplo del directorio /`flow-control`, ya utilizado en el capítulo 12, ilustra cómo lograrlo. El emisor (cliente) envía datos al receptor (servidor) tan rápido como puede, pero el receptor limita la tasa aplicando la técnica anterior. Ambos programas calculan la tasa de envío y recepción respectivamente mediante promedio acumulado (§ 17.2.2).

Cliente

```
~$ ./client.py 127.0.0.1 2000
SNDBUF: 2,626,560 bytes
 (-) sent:14,550.0 kB, CA:245.2 kB/s
```

Servidor

```
~$ ./server.py --limit 200 2000
Client connected: ('127.0.0.1', 42244)
RCVBUF: 131,072 bytes
 received:11,869.8 kB, CA:200.1 kB/s
```

FIGURA 17.1: Limitación de la tasa en el receptor
/flow-control

Puedes probar el ejemplo con los siguientes comandos. El segundo parámetro del servidor es la tasa máxima deseada: 200 kB/s.

Al ejecutarlo, la transferencia en el lado del cliente se detiene durante unos segundos de vez en cuando. El receptor no para en ningún momento porque sigue consumiendo los datos que hay en el buffer de recepción. Cuando se libera suficiente espacio, la ventana se abre y el emisor envía nuevos datos durante un tiempo, hasta que el buffer se llena y la ventana se cierra de nuevo.

La Figura 17.2 es un esquema de su funcionamiento y la Figura 17.3 muestra la cantidad de datos enviados por el emisor a partir de la información obtenida de su ejecución (almacenada en el archivo `client-stats.csv`). El diagrama en escalera permite apreciar claramente los períodos de bloqueo *(stall)* debidos al cierre de la ventana.

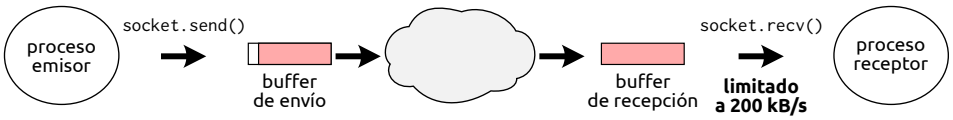


FIGURA 17.2: Limitación de tasa en el receptor

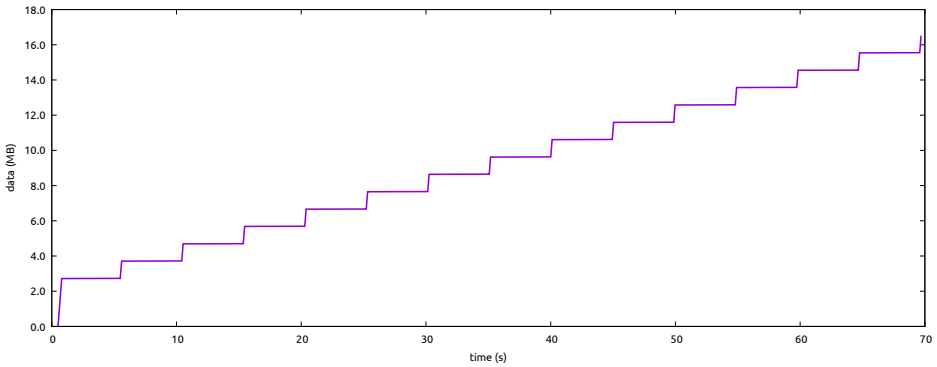


FIGURA 17.3: Datos enviados por el emisor

En la evolución de la tasa se aprecia claramente el efecto del calentamiento o *warmup* (§ 17.2.5). Debido a que los buffers de envío y recepción son grandes: 2627 kB y 131 kB respectivamente[5], el emisor al comienzo puede enviar muchos datos ($\sim$ 2,7 MB) muy rápido. Una vez llenos los buffers, bloquea por el cierre de la ventana. Con el tiempo la tasa media se irá aproximando a los 200 kB/s impuestos por el receptor. Puedes ver esa evolución en la gráfica 17.4 obtenida también a partir de los datos generados por el emisor.

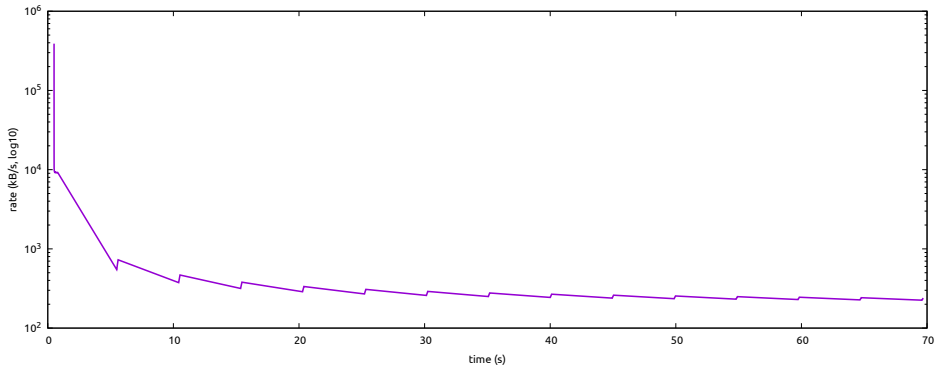


FIGURA 17.4: Tasa de envío medida en el emisor (promedio acumulado)

Los dientes de sierra se deben a que el emisor envía muchos datos en poco tiempo y bloquea (rampa ascendente); cuando reanuda, han pasado unos segundos por lo que la tasa media baja un poco (rampa descendente). Usando otro método para el cálculo de tasa, lógicamente obtendríamos gráficas sin este efecto.

Este mismo servidor proporciona alternativamente un modo diferente de trabajo (que se activa con `--step`). Este argumento permite especificar una cantidad de bytes, que una vez recibidos hacen que el servidor se detenga en espera de que el usuario pulse `ENTER` para continuar. De ese modo puedes comprobar fácilmente cómo el emisor bloquea al llenarse los buffers de envío y recepción y no continúa hasta que quede espacio libre en el receptor. Esta lógica está implementada en el método `step_receiving()` (Listado 17.4).

```
159     def step_receiving(self):
160         input('Press ENTER to receive > ')
161         received = 0
162         while 1:
163             count = 0
164             pending = self.step_size * 1000
```

[5]Esos son los valores por defecto en GNU/Linux para `localhost`.

```
165             while count < pending:
166                 data = self.conn.recv(min(4096, pending - count))
167                 if not data:
168                     return
169                 count += len(data)
170             received += count
171
172             input(f'received: {received//1000:,} kB > ')
```

Listado 17.4: Servidor con control manual de la recepción
/flow-control/server.py

En los ejemplos previos el cliente simplemente envía datos sin sentido, y el servidor los descarta. Sin embargo, tienen otro modo que permite indicar al cliente que consuma datos desde su entrada estándar (con `--stdin`) y al servidor que envíe los datos recibidos a su salida estándar (con `--stdout`). De este modo podemos crear un sistema sencillo para transmitir un fichero `mp3` a través de la red, que será consumido en el servidor directamente por un reproductor de audio, vía redirección. Obviamente para que funcione correctamente, el cliente debe enviar un archivo adecuado (un `mp3`). Este escenario es interesante porque es el reproductor el que determina la tasa de recepción en función de la calidad y compresión del archivo (su *bitrate*). Puedes probar esa posibilidad con estos comandos:

```
~$ ./server.py --stdout --rcvbuf 4000 2000 | mpg123 -q -
~$ ./client.py --stdin --sndbuf 4000 < audio.mp3
```

Figura 17.5: Tasa limitada por el reproductor de `mp3`
/flow-control-player

Fíjate en las opciones `--sndbuf` y `--rcvbuf` que establecen el tamaño de los respectivos buffers de envío y recepción. Estos valores favorecen períodos de bloqueo más cortos que ayudan a entender cómo funciona la transferencia.

Tanto el cliente como el servidor muestran la tasa con promedio acumulado (CA). El servidor además puede mostrar la tasa SMA si se activa el argumento `--sma` y la tasa EMA con compensación de calentamiento basada en el ajuste de α respecto Δt si se activa el argumento `--ema`. La Figura 17.7 representa ambas tasas para comparación.

En la ejecución de la Figura 17.5 la tasa de recepción ronda los 41 kB/s (328 kbps), muy próxima al bitrate con el que está codificado el `mp3` con el que hemos realizado la prueba: 320 kbps.

```
flow-control$$ ./server.py --ema --sma --stdout --rcvbuf 4000 2000 | mpg123 -q -
Client connected: ('127.0.0.1', 52406)
RCVBUF: 8,000 bytes
 received:968.7 kB, CA:43.0 kB/s, EMA:40.8 kB/s, SMA:41.0 kB/s
```

FIGURA 17.6: Opciones para medición de tasa en el servidor: EMA y SMA /flow-control-player/server.py

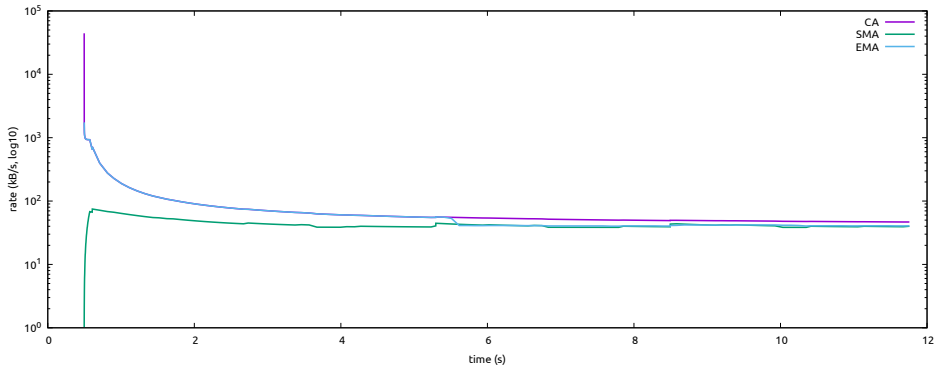


FIGURA 17.7: Comparación de CA, SMA y EMA en el servidor

La consecuencia de tener el reproductor en el receptor es la misma que en los otros ejemplos. Como la tasa de recepción es menor que la tasa de emisión, ambos buffers se llenan y el emisor queda eventualmente bloqueado de manera intermitente. En este caso, también el receptor puede quedar bloqueado porque la entrada estándar del reproductor dispone de un buffer adicional, con lo que la llamada `stdout.flush()` puede bloquear el proceso receptor hasta que haya espacio libre en ese buffer. Ten en cuenta que esto ocurre porque cliente y servidor se están ejecutando en el mismo nodo y se comunican a través de `localhost`. En una conexión remota y en función del ancho de banda disponible, este comportamiento puede variar mucho, hasta el punto de no ocurrir bloqueos si ese ancho de banda es similar al bitrate al que el reproductor consume el flujo.

```
~$ file audio.mp3
audio.mp3: Audio file with ID3 version 2.4.0, contains: MPEG ADTS, layer III, v1,
320 kbps, 44.1 kHz, JntStereo
```

FIGURA 17.8: Información del `mp3` utilizado en la prueba

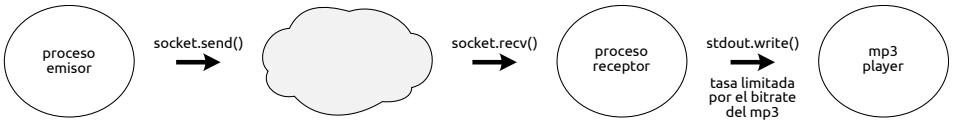


FIGURA 17.9: Datos enviados por el emisor

17.3.2. Limitación de tasa en el emisor

Por supuesto, el emisor puede aplicar exactamente la misma técnica intercalando pausas para conseguir una tasa de emisión concreta. Sin embargo, en este caso la tasa será mucho más estable. El emisor puede enviar bloques de datos más pequeños más a menudo, lo que reduce mucho la dispersión.

El problema es que el emisor no sabe a qué tasa consume el receptor. Si se excede, en algún momento se llenarán los buffers y se bloqueará el emisor. Si es demasiado baja, será el receptor el que se bloquee en espera de datos nuevos y la reproducción del audio se interrumpirá.

17.4. Medición de tasa de transferencia y de pérdida

El programa `iperf3` es una excelente herramienta para medir la tasa de transferencia entre dos nodos y también la tasa de pérdida de paquetes. Se puede utilizar con UDP y TCP con múltiples flujos de datos concurrentes, en uno o en ambos sentidos. Es posible establecer la duración de la prueba tanto en tiempo como en cantidad de datos y fijar la tasa de transferencia deseada, incluso saturando la ruta. La salida del programa muestra tasa de transferencia, latencia y *jitter*.

El programa calcula y muestra la tasa de transferencia instantánea en intervalos de 1s (por defecto). Al acabar muestra el promedio acumulado de toda la prueba.

Este ejemplo muestra las medidas que realizan tanto el servidor como el cliente para un flujo UDP con una tasa solicitada de 50 Mbps, que claramente la ruta puede satisfacer. Los datos van únicamente del cliente al servidor.

```
alice@sidney:~$ iperf3 --server
Accepted connection from 203.0.113.2, port 57850
[  5] local sidney.example.net port 5201 connected to 203.0.113.2 port 47103
[ ID] Interval      Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   5.76 MBytes  48.3 Mbps  0.113 ms  0/4196 (0%)
[  5]   1.00-2.00   5.96 MBytes  50.0 Mbps  0.114 ms  1/4339 (0.023%)
[  5]   2.00-3.00   5.96 MBytes  50.0 Mbps  0.152 ms  0/4340 (0%)
[  5]   3.00-4.00   5.96 MBytes  50.0 Mbps  0.078 ms  0/4339 (0%)
[  5]   4.00-5.00   5.96 MBytes  50.0 Mbps  0.093 ms  0/4341 (0%)
[  5]   5.00-5.03    207 KBytes  50.1 Mbps  0.175 ms  0/147 (0%)
```

```
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.03   29.8 MBytes  49.7 Mbps  0.175 ms  1/21702 (0.0046%) receiver
```

```
$ iperf3 --time 5 --bitrate 50M --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 47103 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   5.97 MBytes  50.0 Mbps  4345
[  5]   1.00-2.00   5.96 MBytes  50.0 Mbps  4340
[  5]   2.00-3.00   5.96 MBytes  50.0 Mbps  4341
[  5]   3.00-4.00   5.96 MBytes  50.0 Mbps  4340
[  5]   4.00-5.00   5.96 MBytes  50.0 Mbps  4337
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   29.8 MBytes  50.0 Mbps  0.000 ms  0/21703 (0%)  sender
[  5]   0.00-5.03   29.8 MBytes  49.7 Mbps  0.175 ms  1/21702 (0.0046%) receiver
```

Para cada intervalo, aparece la cantidad de datos enviados o recibidos y la tasa media. Al terminar, muestra la cantidad total y la media de toda la prueba. Al terminar, el servidor envía al cliente los datos de resumen, por eso el cliente muestra una línea `sender` y otra `receiver`. Únicamente se perdió un datagrama UDP (última línea).

En esta segunda prueba, el bitrate solicitado es mucho mayor: 1 Gbps. La ruta no puede satisfacer esa demanda, se queda en 90,6 Mbps, y por eso se pierden muchos datagramas, concretamente el 90 %.

```
$ iperf3 --time 5 --bitrate 1G --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 55549 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   114 MBytes   957 Mbps  83134
[  5]   1.00-2.00   114 MBytes   956 Mbps  82973
[  5]   2.00-3.00   114 MBytes   956 Mbps  82968
[  5]   3.00-4.00   114 MBytes   956 Mbps  82976
[  5]   4.00-5.00   114 MBytes   956 Mbps  83013
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00    570 MBytes   956 Mbps  0.000 ms  0/415064 (0%)  sender
[  5]   0.00-5.28   57.1 MBytes  90.6 Mbps  0.190 ms  373469/415040 (90%)  receiver
```

Y finalmente la misma prueba con TCP. En este caso también se pierden paquetes, pero TCP se encarga de retransmitirlos (columna `Retr`), en total 86. Le hemos establecido un intervalo de 0,1 s para poder apreciar mejor el crecimiento de la ventana de congestión (`Cwnd`) aunque claramente los RTO la han dejado cerca de 350 ms. La tasa media de transferencia es de 80,5 Mbps, muy inferior a la solicitada de 1 Gbps, y menor también que la obtenida con el cliente UDP (80,2 vs. 90,6 Mbps) aunque el nodo remoto y probablemente la ruta son los mismos. Esta diferencia se debe sin duda a la contención que provoca la ventana de congestión.

```
$ iperf3 --time 5 --bitrate 1G --client sidney.example.net --interval 0.1
Connecting to host sidney.example.net, port 5201
[ ID] Interval      Transfer     Bitrate    Retr  Cwnd
[  5]   0.00-0.10    128 KBytes  10.5 Mbps    0   56.2 KBytes
[  5]   0.10-0.20   2.50 MBytes   210 Mbps   13    356 KBytes
[  5]   0.20-0.30   0.00 Bytes      0 bps    73    277 KBytes
[  5]   0.30-0.40    768 KBytes  62.9 Mbps    0    283 KBytes
```

```
[  5]   0.40-0.50    768 KBytes  62.9 Mbps    0    288 KBytes
[...]
[  5]   4.80-4.90   1.50 MBytes   126 Mbps    0    394 KBytes
[  5]   4.90-5.00    896 KBytes  73.2 Mbps    0    394 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Retr
[  5]   0.00-5.00   49.8 MBytes  83.5 Mbps   86            sender
[  5]   0.00-5.03   48.1 MBytes  80.2 Mbps                 receiver
```

17.5. Garantizar prestaciones

Lo primero que debes saber es que eso de «garantizar» es bastante relativo. Lo que puede hacer QoS en una interred IP es priorizar un flujo sobre otro, pero la garantía solo la puede dar si el enlace es capaz de ofrecer las prestaciones totales solicitadas o bien se dispone de algún mecanismo de control de admisión.

Hay dos formas básicas de aplicar prioridades al tráfico:

- IntServ (Integrated Services) identifica flujos y aplica prioridades diferentes a cada flujo.
- DiffServ (Differentiated Services) utiliza un conjunto de clases de tráfico predefinidas, marca el tráfico con esas clases y después prioriza los recursos en función de ellas.

IntServ realiza reserva de recursos en cada router antes de comenzar la transmisión. Debido a ello, su escalabilidad es limitada. Los routers que deban manejar miles de flujos deberán mantener información sobre la reserva de recursos de cada uno de ellos. DiffServ es un enfoque más general y escala mucho mejor porque no requiere configuración previa, el router toma sus decisiones únicamente con la información del paquete.

17.6. Servicios diferenciados

Como cabría esperar, la operación de DiffServ ocurre principalmente en los routers, aunque los conmutadores y los equipos terminales también pueden participar en determinadas situaciones. Un esquema simplificado del procesamiento *(pipeline)* QoS podria ser el de la Figura 17.10.

La finalidad de cada una de esas tareas es la siguiente:

- Clasificación: determina a qué clase de tráfico corresponde el paquete.
- Marcado: escribe el identificador de clase en el paquete.
- Policing: comprueba si el paquete cumple con los requisitos de QoS.
- Encolado: coloca el paquete en la cola que corresponde a su clase.

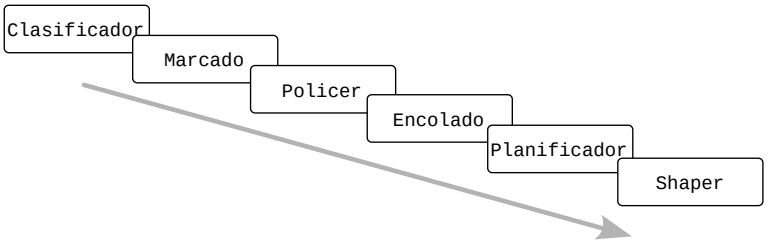


FIGURA 17.10: Pipeline QoS

- Planificación: decide de qué cola se extrae el siguiente paquete a reenviar.
- Shaping: ajusta la tasa de salida conforme a la contratada.
- Reenvío: transmite el paquete hacia su siguiente salto como en un router convencional.

Antes de entrar en el detalle de cada una de estas tareas, conviene introducir dos conceptos que aparecen de forma recurrente en redes, y especialmente en QoS:

ingress es el tráfico que entra.
egress es el tráfico que sale.

La interpretación de ingress y egress dependen completamente del componente que se tome como referencia. El mismo tráfico puede ser una cosa u otra en función de cada caso. Para el pipeline QoS, ingress es el tráfico que procede de la red (si es un router), o de un proceso emisor si es un nodo. El tráfico egress es el que envía a la interfaz de red.

Dentro del propio pipeline, las colas donde se almacenan temporalmente los paquetes son el punto de referencia más intuitivo. Asi, las tareas de clasificación, marcado y policing procesas el tráfico ingress, mientras que planificación, shaping y reenvío se encargan del tráfico egress.

17.6.1. Cubos de tokens

Para saber si un flujo cumple una determinada tasa (o forzar que así sea) se suelen utilizar variantes del algoritmo de medición *token bucket*, que podemos traducir como *cubo de fichas* o simplemente *cubo de tokens*.

El cubo[6] se llena de tokens a una tasa constante de CIR, que representa la tasa media garantizada. El cubo tiene una capacidad de CBS, que representa el tamaño de ráfaga máxima permitida. Cada paquete que llega se

[6] ficticio, obviamente

canjea por tokens del cubo (un token por byte). Si el cubo contiene tokens suficientes, el paquete es conforme y sigue su camino. En caso contrario, se dice que el paquete «excede» y se aplica la acción programada que decida el servicio que está usando el algoritmo. Si eso ocurre, no se consumen tokens.

El algoritmo del cubo de tokens deja pasar los paquetes si su tasa media fluctúa en torno a CIR, pero además tolera ráfagas ocasionales superiores siempre que compensen los tokens no consumidos (ahorrados) durante la emisión por debajo de CIR. Esta es la versión básica y se conoce como *single token bucket* o *single rate two color marker*. Esos dos colores representan el resultado: conforme (verde) o excedente (rojo). La Figura 17.11 representa el algoritmo.

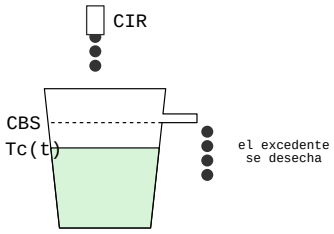


FIGURA 17.11: Cubo de tokens básico: dos colores

La Tabla 17.1 es un ejemplo del funcionamiento de este cubo de tokens. El cubo se llena a una tasa CIR=1000 kBps y tiene una capacidad CBS=3000 bytes. Inicialmente el cubo está lleno ($T_c(t) = 3000$ tokens). El paso (Δt) es 1 ms, es decir, cada fila muestra la llegada de un paquete con esa diferencia de tiempo. Las columnas «$T_c(t)$ prev.» y «$T_c(t)$ post.» son los tokens acumulados en el cubo antes y después de evaluar cada paquete; B es el tamaño del paquete entrante (bytes) y «Resultado» es el color con el que se marca el paquete. Por último, la columna «media» muestra la tasa media acumulada (bytes recibidos / tiempo transcurrido). Se mantiene en torno a los 1000 kBps aunque la tasa de entrada suba hasta los 1800 kBps; esto es una ráfaga y se puede ver que si dura demasiado, el cubo la corta.

La variante SRTCM (Single Rate Three Color Marker) [54][7] utiliza dos cubos `C` *(Commited)* y `E` *(Exceed)*. El cubo `C` tiene una capacidad de CBS tokens que juega el mismo papel que en el algoritmo básico. El cubo `E` tiene una capacidad de EBS, se llena con el excedente de `C` y representa el tamaño de ráfaga excedente (son tokens ahorrados). El paquete se marca como

[7] Estamos aplicando el modo *color blind*, es decir, los paquetes llegan sin colorear.

$t(ms)$	$T_c(t)$ prev.	B	$T_c(t)$ post.	Resultado	media
0	3000	1000	2000	verde	1000,0
1	3000	1200	1800	verde	1100,0
2	2800	700	2100	verde	966,7
3	3000	1800	1200	verde	1175,0
4	2200	1800	400	verde	1300,0
5	1400	1800	1400	**rojo**	1083,3
6	2400	1800	600	verde	1185,7
7	1600	1800	1600	**rojo**	1037,5
8	2600	1000	1600	verde	1033,3

CUADRO 17.1: Ejemplo de cubo de tokens

«conforme» (verde) si los tokens acumulados en `C` ($T_c(t)$) son suficientes. Si no lo son, pero los tokens acumulados en `E` ($T_e(t)$) son suficientes, se marca como «excedente» (amarillo). En otro caso se marca como «fuera de perfil» (rojo). El paquete marcado en verde consume tokens de `C`, el marcado en amarillo consume tokens de `E` y el marcado en rojo no consume tokens. La Figura 17.12 representa el algoritmo y el Listado 17.5 muestra su pseudocódigo.

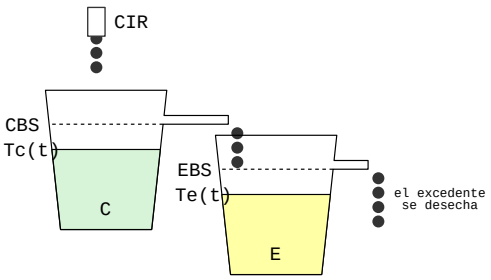


FIGURA 17.12: Algoritmo de cubo de tokens de tipo srTCM

```
B = paquete.longitud
si Tc >= B:
    color = VERDE
    Tc = Tc - B
sino si Te >= B:
    color = AMARILLO
    Te = Te - B
sino:
    color = ROJO
```

LISTADO 17.5: Algoritmo de srTCM en pseudocódigo

Una segunda variante, llamada TRTCM (Two Rate Three Color Marker) [55], utiliza dos tasas de llenado: CIR y PIR para los cubos `C` *(Commited)* y `P` *(Peak)*, que tienen capacidades CBS y PBS respectivamente. Para un paquete de B bytes, marca en rojo si `P` tiene menos de B tokens, amarillo si `P` los tiene, pero `C` no, y verde si ambos los tienen. Cuando se marca verde, se descuentan B tokens de ambos cubos. El amarillo indica que el paquete excede la ráfaga máxima garantizada (CBS), pero no la máxima permitida (PBS). Se representa en la Figura 17.13.

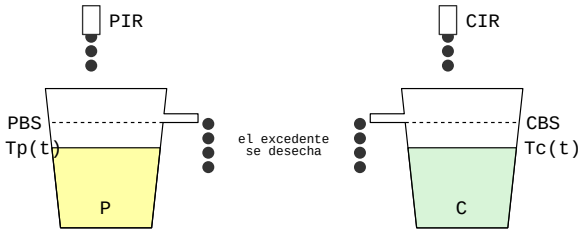


FIGURA 17.13: Algoritmo de cubo de tokens de tipo TRTCM

Estos algoritmos solo miden. El uso que se le dé depende de la acción que se aplique para cada color resultante: aceptar, descartar, remarcar, esperar, enviar a otra cola, etc.

17.6.2. Clasificación y marcado

El marcado debería realizarse lo más cerca posible del origen del tráfico. Sin embargo, normalmente no se hace en el propio origen, a menos que se trate de nodos bajo el control de la organización o terminales cerrados como teléfonos VOIP. Piensa que si cualquiera pudiera marcar su tráfico en su propio computador, todos los usuarios elegirían la máxima prioridad y todo el sistema sería inútil. Los dispositivos en los que sí se confía —routers, switches u otros nodos bajo control de la organización— forman la «frontera de confianza» *(trust boundary)*.

Cuando el marcado lo hace un router se realiza considerando los datos de la cabecera del paquete: dirección origen y destino, puerto, protocolo, etc., pero también información de enlace como la etiqueta VLAN, por lo que todos los paquetes de un mismo flujo se marcan probablemente con la misma clase. Si el paquete llega marcado, el router podría decidir respetar la marca o sobreescribirla si no se considera fiable.

El marcado del tráfico consiste en colocar un identificador de clase —un valor de 6 bits llamado PHB (Per-Hop Behavior)— en el campo DSCP de la cabecera IP. En IPV6, el campo `Traffic Class` cumple la misma función.

El campo `DSCP` ocupa los 6 bits MSB del campo TOS (Type of Service) de la cabecera; los 2 bits restantes se utilizan para ECN, la notificación de congestión (§ 14.6).

El significado de ese octeto ha cambiado con cada especificación que lo ha revisado. La Figura 17.14 compara esas interpretaciones, una por fila:

- La RFC 791 [9] (1981), la especificación original de IP, lo definió como campo `TOS`: tres bits de precedencia y tres indicadores independientes para solicitar menor retardo (D:*delay*), mayor tasa (T:*throughput*) o mayor fiabilidad (R:*reliability*), con los dos últimos bits reservados a cero.
- La RFC 1349 [56] (1992) añadió un cuarto indicador, el coste económico (C:*cost*), que ocupó uno de esos bits reservados.
- La RFC 2474 [57] (1998) abandonó ese esquema y reinterpretó los seis bits más significativos como un único valor, el DSCP, dejando los dos bits restantes sin asignar; la RFC 3168 [51] (2001) los destinó a ECN.
- La RFC 2597 [58] (1999) da estructura interna al propio DSCP: para las clases AFxy reparte sus seis bits en tres de clase, dos de tolerancia al descarte y uno fijo a cero. Aunque es anterior a la RFC 3168, ocupa el último lugar porque no reinterpreta el octeto completo, sino que subdivide el DSCP que definió la RFC 2474.

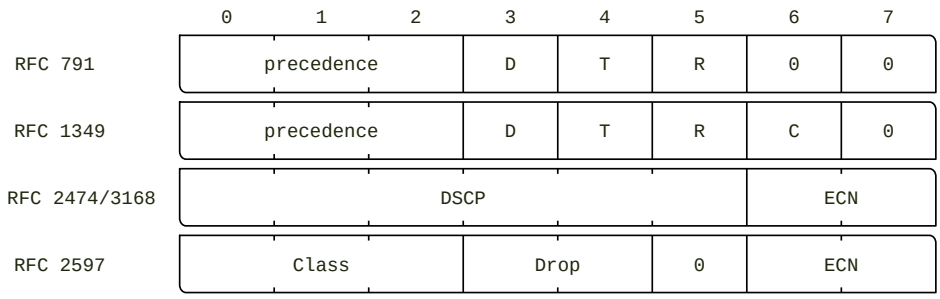


FIGURA 17.14: Interpretaciones del campo `TOS` de IP

Los PHB estandarizados son:

AC (Class Selector)
: (bits `xxx000`) indica una clase de tráfico codificada en los tres bits MSB (CS1-CS7). DF es en realidad la clase 0 (CS0). Estas clases están definidas en la RFC 2474 [57] y son compatibles con el campo `Precedence` de la especificación original de IP RFC 791 [9].

Las clases, de menor a mayor prioridad, son:

CS1 Tráfico desechable. De hecho, tiene menor prioridad que CS0, *p. ej.* P2P, actualizaciones en segundo plano.

CS0 Tráfico sin garantías *best effort*, la clase por defecto.

CS2 OAM (Operations, Administration and Maintenance), *p. ej.* SSH, SNMP, NTP.

CS3 Video broadcast, *p. ej.* IPTV.

CS4 Vídeo interactivo en tiempo real, *p. ej.* videojuegos, telemetría, telepresencia.

CS5 Señalización de voz y vídeo, *p. ej.* SIP, H.323, H.245.

CS6 Control de la red y encaminamiento, *p. ej.* OSPF, BGP, STP.

CS7 Control crítico / reservado.

AFxy (Assured Forwarding)
(bits `xxx100`) indica una de cuatro clases con tres niveles de descarte. Se representan como AFxy, siendo x la prioridad de reenvío (1-4) e y la tolerancia que ese tráfico tiene al descarte (1-3). Por ejemplo, AF41 indica alta prioridad (4) y baja tolerancia al descarte (1).

EF (Expedited Forwarding)
(bits `101110`) es el tráfico de máxima prioridad, RTP/VoIP.

Estos modos diferentes de codificar las clases en realidad se intercalan. La RFC 4594 [59] define un conjunto de 12 perfiles de tráfico, que puedes ver resumidos en el Cuadro 17.2.

Clase de tráfico	PHB / DSCP	Uso típico
Control de red	CS6	Protocolos de encaminamiento
Telefonía	EF	Voz sobre IP
Señalización	CS5	Señalización de llamadas
Videoconferencia	AF41/42/43	Videoconferencia interactiva
Interactivo en tiempo real	CS4	Vídeo interactivo, juegos en red
Streaming multimedia	AF31/32/33	Streaming de vídeo y audio
Vídeo broadcast	CS3	Vídeo broadcast en tiempo real
Datos de baja latencia	AF21/22/23	Transacciones interactivas (ERP, BD)
OAM	CS2	Gestión de red
Datos de alto rendimiento	AF11/12/13	Transferencias masivas, correo, copias de seguridad
Estándar	DF / CS0	Tráfico genérico
Datos de baja prioridad	CS1	Tráfico desechable

CUADRO 17.2: Clases de tráfico recomendadas

17.6.3. Policing

El policing (en español «vigilancia» o «regulación») comprueba que el tráfico no excede los valores de tasa (CIR/PIR) establecidos en el SLA. Aunque evalúa paquetes individuales, tiene en cuenta el comportamiento acumulado de cada flujo. Si el paquete analizado hace que el flujo exceda la tasa prevista, puede descartarlo o remarcarlo con una prioridad inferior o preferencia de descarte mayor, por ejemplo cambiando un marcado AF41 a AF43.

El policer a menudo utiliza un medidor de cubo de tokens de tres colores (srTCM o trTCM) para determinar si el paquete es conforme (verde), excedente (amarillo) o fuera de perfil (rojo). Las acciones que aplica el policer suelen ser las siguientes:

- Si el paquete es conforme, lo deja pasar sin cambios.
- Si el paquete excede, lo deja pasar, pero lo marca con una prioridad inferior o una preferencia de descarte mayor.
- Si el paquete está fuera de perfil, lo descarta.

Con estas acciones puedes deducir que el policer no añade latencia ni *jitter* al flujo; nunca encola ni retrasa, pero como descarta, puede provocar retransmisiones. Por eso, el policer se aplica sobre todo con tráfico prioritario transportado sobre UDP, como VoIP, en el que es preferible perder algún pequeño fragmento que retrasar todo el flujo.

17.6.4. Encolado

A cada interfaz de salida del router se asocia un conjunto de colas (una por clase) en lugar de una sola (§ 14.1). Es una tarea sencilla, simplemente se coloca cada paquete en la cola que le corresponde según la clase con la que está etiquetado.

Los paquetes quedan almacenados en estas colas en espera de que el canal quede libre y el router pueda reenviar el siguiente paquete. Al igual que en un router sin QoS, la ocupación de estas colas crece durante episodios de alta carga. En esa situación (ver § 14.6) se utiliza un algoritmo de descarte aleatorio (RED) en lugar de esperar a que la cola se llene.

Sin embargo, con un router QoS, la situación es distinta porque disponemos de información adicional sobre el tráfico. Se aplican algoritmos como WRED (Weighted RED), que considera tanto la clase del paquete como su tolerancia al descarte (la y en las clases AFxy) y que elige paquetes cuyo descarte resulta menos dañino.

Como norma, no se aplica WRED a la cola de máxima prioridad (EF) porque ese tipo de tráfico (*p. ej.* VoIP) tolera muy mal el descarte. Como suele trasportarse sobre UDP no se retransmite, no soporta ECN ni su descarte ayuda a reducir la congestión.

17.6.5. Planificación

El planificador *(scheduler)* es el algoritmo utilizado para determinar de qué cola se toma el siguiente paquete. Estos son los tipos de planificador más comunes, en orden de complejidad:

FIFO
: (First In First Out) representa la ausencia de prioridad. Todos los paquetes son tratados del mismo modo, y se reenvían en el orden de llegada. Se podría decir que es el planificador de los routers sin QoS.

SP
: (Strict Priority) como su nombre indica, aplica una prioridad estricta, es decir, siempre atiende primero a la cola no vacía de mayor prioridad. Muy simple, pero produce inanición para las colas de menor prioridad en episodios de alta carga.

  Al atender siempre primero la cola de mayor prioridad, un paquete de esa clase nunca espera detrás de tráfico de otra. Así acota el retardo máximo de la clase y, al reducir su competencia por la cola, también el *jitter* (§ 17.1.1).

WRR
: (Weighted Round Robin) reparte el ancho de banda disponible en el enlace entre las colas proporcionalmente a su prioridad.

WFQ
: (Weighted Fair Queuing) es similar a WRR pero considera el tamaño de los paquetes para determinar la proporción de ancho de banda consumida por cada cola. De ese modo, una cola de baja prioridad con paquetes grandes no distorsiona la proporción asignada.

CBWFQ
: (Class Based WFQ) es un WFQ configurable. Permite al administrador definir las clases y el reparto del ancho de banda, garantizando a cada una una tasa mínima incluso cuando el resto de clases satura el enlace.

LLQ
: (Low Latency Queuing) es el planificador más usado. Es una combinación de SP para tráfico de alta prioridad y baja latencia, y CBWFQ para el resto del tráfico. SP se aplica hasta una tasa máxima configurable, para evitar inanición.

La cota que SP impone al retardo es fácil de calcular. Como es un planificador no apropiativo *(non-preemptive)*, no puede interrumpir el envío un paquete en curso aunque llegue otro de mayor prioridad, pero cuando acaba atiende de inmediato la cola prioritaria. Ese es el peor caso y corresponde con el retardo de envío de § 17.1.1 aplicado al mayor paquete posible:

$$\text{retardo}_{\max} \approx \frac{L_{\max}}{R} \tag{17.8}$$

donde:

> $L_{\max}$ es el tamaño del mayor paquete que puede estar enviándose en el enlace —seguramente MTU—, y R la velocidad de transmisión del enlace.

Por ejemplo, en un enlace Gigabit Ethernet ($R = 1$ Gbps) con tramas de hasta 1500 bytes, el retardo adicional es como mucho de $1500{\cdot}8/10^9 = 12\ \mu s$, un valor insignificante frente al orden de milisegundos en el que se suele definir un SLA de voz o vídeo. Lógicamente la cota crece en enlaces más lentos porque el mismo paquete tarda más en enviarse.

17.6.6. Shaping

El *shaping* (en español «modelado» o «conformado») es la tarea que extrae paquetes de la cola elegida por el planificador. Decide el momento adecuado para obtener una tasa de salida estable en torno al CIR configurado, y para eso utiliza también el algoritmo de cubo de tokens, aunque no exactamente del mismo modo que el policer. Mientras el cubo contenga tokens suficientes, el shaper los canjea por paquetes de la cola; si no hay, espera, bloquea la cola.

Este cubo de tokens se configura con tasa de llenado CIR y capacidad CBS. Con estos valores se calcula $\Delta t = CBS/CIR$, que es el período de extracción. Un CBS alto produce Δt grandes, lo que conlleva ráfagas grandes y períodos largos; un CBS bajo conlleva ráfagas pequeñas y períodos cortos. Por ejemplo, si la tasa garantizada es $CIR = 6Mbps$ y $CBS = 8Kb$ obtenemos un $\Delta t = 1{,}33$ ms. Eso significa que el shaper extrae 8 Kb cada 1,33 ms. Si para el mismo caso se fija $CBS = 800Kb$ se extraerán esos 800 Kb cada 133 ms. Ese mismo Δt es también el límite superior del *jitter* que el propio shaping puede introducir en la clase: un CBS pequeño lo mantiene bajo, a costa de admitir ráfagas menores.

El valor Δt es el período del reloj que controla la acción del shaper, pero también del planificador. También aquí se puede utilizar un algoritmo de dos cubos con capacidades CBS y EBS del tipo SRTCM.

17.7. QoS en Linux

La mayoría de la «maquinaria» de QoS de Internet se encuentra en equipos de gama alta, como switches y routers en grandes empresas, pero sobre todo en ISP y en los grandes operadores de telecomunicaciones en las redes WAN. El fabricante CISCO destaca por su sofisticado soporte de QoS en este tipo de equipamiento con su sistema operativo IOS.

Linux también dispone de un buen soporte de QoS, que puede ser suficiente para despliegues de mediano y pequeño tamaño, y para el propósito de este libro resulta muy útil para bajar al detalle, experimentar y comprobar en la práctica lo explicado hasta ahora.

El soporte de QoS de Linux se llama *Traffic Control*, o simplemente `tc` (`iproute2`), que es también el nombre del programa con el que se administran la mayoría de sus opciones. Aunque la nomenclatura de Linux difiere un poco de la teoría de las secciones anteriores (extraída principalmente de las RFC), los conceptos y la forma en que funciona son muy similares. *Traffic Control* se apoya en tres conceptos:

qdisc
: de *queuing discipline*, que podemos traducir como «disciplina de encolado», es equivalente al planificador.

class
: son las clases de tráfico. Cada clase implica una cola, pero en función del tipo de qdisc, pueden ser varias.

filter
: son los predicados que indican a qué clase debe ir cada paquete. El conjunto de filtros forma el clasificador.

Los siguientes apartados explican cómo se configuran estos tres elementos y cómo se relacionan entre sí.

17.7.1. qdisc

Aunque es probable que lo hayas pasado por alto hasta ahora (y nosotros lo hemos omitido en muchas capturas del libro) cada interfaz de red puede tener asociada una qdisc. El comando `ip link` lo muestra de nuevo:

```
$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP qlen 1000
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

```
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 08:00:46:de:ca:f0 brd ff:ff:ff:ff:ff:ff
```

Los valores de qdisc para cada interfaz:

```
lo: qdisc noqueue
eth0: qdisc fq_codel
wlan0: qdisc noqueue
```

La interfaz Ethernet está configurada con el qdisc `fq_codel` —la opción por defecto hoy día. Sin embargo, las interfaces *loopback* y WiFi tienen configurado `noqueue`, que es como decir 'sin qdisc', pero es así por motivos diferentes.

Loopback no es una interfaz real, no está asociada a un medio físico, o sea que no puede sufrir congestión y los flujos que la utilizan no compiten por su ancho de banda. La interfaz WiFi por otra parte, sí que utiliza QoS, pero lo hace en una capa inferior. WiFi utiliza un medio compartido (el aire) por el que los dispositivos tienen que competir, y lo hacen con un sistema llamado CSMA/CA. Es la capa MAC, que está implementada en la propia NIC, la que se encarga en gran medida de afrontar el problema. Ya tiene sus propias colas y su lógica de prioridad como parte de la implementación del protocolo de enlace. A pesar de todo eso, puede haber situaciones en las que interese completar las capacidades QoS desde el SO, pero no es lo habitual.

El Cuadro 17.3 muestra algunos de los qdisc disponibles, y su relación con los tipos de QoS:

`pfifo` y `bfifo`	FIFO con conteo de paquetes y bytes
`prio`	SP con varias bandas
`sfq`	WRR
`htb`	WFQ y CBWFQ

CUADRO 17.3: Equivalencia entre qdisc y los tipos de QoS

Esta es la configuración por defecto en un Linux actual de una interfaz Ethernet, con el comando `tc`:

```
~# tc qdisc show dev eth0
qdisc fq_codel 0: root refcnt 2 limit 10240p flows 1024 quantum 1514 target 5ms
    interval 100ms memory_limit 32Mb ecn drop_batch 64
```

Esta interfaz utiliza `fq_codel`, que combina dos mecanismos: FQ *(fair queuing)*, un clasificador que utiliza un hash para direcciones/puertos/protocolo/etc. y con ello organiza los flujos de tráfico en un número especificado de colas

internas. Sirve esas colas con un planificador de tipo *round robin* similar a WRR/DRR para hacer un reparto equitativo (de ahí su nombre) de la tasa excedente entre esas colas. CoDel *(controlled delay)* es la técnica de AQM que aplica en caso de congestión (capítulo 14, § 14.6.1). A diferencia de SP/LLQ (§ 17.6), que acotan el retardo dando prioridad completa a una clase, CoDel trata igual a todas las colas internas de `fq_codel`: es el control de retardo que aplica Linux por defecto actualmente.

Este comando ofrece mucha información útil que requiere explicación:

`0: root`
: Dice que es el planificador raíz o principal. Más adelante en este mismo capítulo se muestra cómo definir jerarquías de planificadores y colas.

`limit 10240p`
: Es el tamaño de las colas internas: 10240 paquetes cada una.

`flows 1024`
: Es el número de colas internas. Fíjate que puede haber más flujos que colas. La cola se comparte entre varios flujos si tienen el mismo valor de hash.

`quantum 1514`
: Es la cantidad de bytes que cada cola puede transmitir en cada ronda antes de ceder el turno a la siguiente cola (DRR). El valor de 1514 bytes no es casual; es justo el tamaño de una trama Ethernet con su cabecera. La idea es que se pueda extraer al menos un mensaje por ronda y no haya que esperar al siguiente turno para acumular déficit.

`target 5ms interval 100ms`
: Es configuración para CoDel: `target` es el retardo aceptable de los paquetes. Por encima de ese valor, se considera su descarte; `interval` es el plazo máximo que el flujo tiene permitido superar `target`. En este caso, si el retardo del flujo se mantiene por encima de 5 ms durante 100 ms, se empezarán a descartar paquetes o se aplicará ECN si está disponible.

`memory_limit 32Mb`
: Es el espacio total (en megabytes) que ocupan los paquetes encolados (los 10240 paquetes de `limit` en este ejemplo).

`ecn` ECN está habilitado.

`drop_batch 64`
: La cantidad de paquetes que se pueden descartar de una vez si se supera `limit` o `memory_limit`. Solo se hace en caso extremo como una forma de tail-drop eficiente.

El programa `tc` ofrece estadísticas detalladas sobre el comportamiento de la política QoS establecida en la interfaz[8]. Para la interfaz anterior, nos dice lo siguiente:

```
$ tc -s qdisc show dev eth0
 Sent 1879915814 bytes 2378552 pkt (dropped 0, overlimits 0 requeues 8993)
 backlog 130204b 2p requeues 8993
  maxpacket 65102 drop_overlimit 0 new_flow_count 3065 ecn_mark 0 memory_used 126448
  new_flows_len 0 old_flows_len 1
```

[8]Las cifras mostradas están contabilizadas desde que se definió el planificador.

donde:

`Sent 1879915814 bytes 2378552 pkt`
: Es la cantidad de bytes y paquetes que han sido transmitidos a través de este pipeline.

`dropped 0`
: Paquetes descartados debidos a la congestión por el algoritmo AQM.

`overlimits 0`
: Se superó la tasa garantizada. Con `fq_codel`, este valor siempre es 0 porque no tiene tasa máxima.

`requeues 8993`
: Paquetes reencolados, normalmente por problemas esporádicos, no por congestión o exceso de tasa.

`backlog 130204b 2p`
: Cantidad de bytes y paquetes que hay en la cola en el momento de ejecutar el comando.

`maxpacket 13554`
: El tamaño del paquete más grande que se ha enviado.

`drop_overlimit 0`
: Cantidad de paquetes descartados por superar los límites de la cola (`limit` o `memory_limit`). Es decir, son descartes *tail drop* por congestión.

`new_flow_count 2680`
: Cantidad de flujos que han sido encolados.

`ecn_mark 0`
: Cantidad de paquetes marcados con ECN por congestión.

`memory_used 126448`
: Tamaño total (en bytes) que ocupan los paquetes encolados.

`new_flows_len 0`
: Cantidad de flujos nuevos, que no han sido atendidos aún.

`old_flows_len 1`
: Cantidad de flujos antiguos, de los cuales ya se han atendido algunos paquetes.

Para obtener una salida como la del comando anterior, con datos algo más interesantes, hemos forzado tráfico sintético con el programa `iperf3` (Figura 17.15). Consiste en un servidor (por defecto escucha en el puerto 5201) y un cliente que le envía tráfico. Si no se indica otra cosa, creará conexiones TCP y transmitirá datos a la tasa máxima posible.

Cliente

```
$ iperf3 --time 20 --client
↪  sidney.example.net
    --parallel 4
```

Servidor

```
$ iperf3 --server
```

FIGURA 17.15: Generando tráfico con `iperf3`:
4 flujos TCP en paralelo durante 20 segundos

Aquí en concreto le hemos pedido que envíe tráfico con 4 conexiones en paralelo durante 20 segundos. Cuando acaba, tanto el servidor como el cliente muestran unas estadísticas sobre la transferencia (Listado 17.6).

```
~$ iperf3 --time 3 --client sidney.example.net --port 5201
Connecting to host example.net, port 5201
[  5] local 192.168.0.37 port 53204 connected to example.net port 5201
[ ID] Interval      Transfer     Bitrate    Retr  Cwnd
[  5]   0.00-1.00   6.88 MBytes  57.6 Mbps  103    208 KBytes
[  5]   1.00-2.00   6.38 MBytes  53.4 Mbps    0    224 KBytes
[  5]   2.00-3.00   6.25 MBytes  52.4 Mbps    0    246 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Retr
[  5]   0.00-3.00   19.5 MBytes  54.5 Mbps  103        sender
[  5]   0.00-3.04   17.9 MBytes  49.4 Mbps             receiver
```

Listado 17.6: Estadísticas de transferencia de `iperf3`

17.7.2. Configurando un sistema de colas

El planificador `fq_codel` resulta muy adecuado para las necesidades cotidianas y por eso es la configuración por defecto en muchas distribuciones GNU/Linux. Pero hay situaciones con necesidades específicas. En esos casos, puede ser más adecuado un planificador que permita crear clases y jerarquía de colas. Además nos viene bien para experimentar y aprender de QoS.

Empezamos por configurar un planificador `htb` como raíz, sustituyendo así el `fq_codel` por defecto. Este planificador es de tipo CBWFQ, dos cubos de tokens con tasa garantizada y máxima, similar a srTCM.

```
~# tc qdisc add dev eth0 root handle 1: htb default 99
```

donde:

`root handle 1:`
: Es el identificador del planificador raíz.

`htb` Es el tipo de planificador.

`default 99`
: Es la clase por defecto, que se utilizará para marcar los paquetes que no cumplan ningún filtro.

Definimos una jerarquía de clases sencilla: una clase padre y dos hijas. Eso permite una gestión flexible del ancho de banda disponible entre las clases hermanas. En primer lugar, la clase padre:

```
~# tc class add dev eth0 parent 1: classid 1:1 htb rate 2mbit ceil 2mbit
```

donde:

`parent 1:`
: Es una clase hija del planificador `1:`.

classid 1:1
: Es el identificador de esta clase. El *classid* es un entero de 32 bits con el formato *major:minor*. El *major* es el identificador de la qdisc y el `minor` es el identificador de la clase dentro de esa qdisc. Por eso todo lo que definamos aquí comparte el `major 1:` del qdisc raíz.

rate 2mbit
: Es la tasa garantizada (2 Mbps).

ceil 2mbit
: Es la tasa máxima permitida (2 Mbps).



Es conveniente aclarar que la nomenclatura de `tc` para las unidades es bastante «pintoresca», lo que puede dar lugar a confusiones. Las cantidades en bytes se denotan con una '`b`' minúscula (*p. ej.* `1000b`) mientras que para bits se escribe la palabra completa (`1000 bits`). También se utiliza arbitrariamente «mbits» o «Mbits» para megabits. Y en muchas ocasiones las tasas se expresan como `mbit` para indicar `Mbps` (megabits por segundo). Revisa el anexo D para ver la nomenclatura del SI que se usa en este libro.

Dos clases hijas idénticas (`1:10` y `1:20`) completan la jerarquía y van a compartir la tasa de la clase padre. Para que el tráfico convencional no afecte a los resultados del experimento, se define la clase por defecto (`1:99`).

```
~# tc class add dev eth0 parent 1:1 classid 1:10 htb rate 1mbit ceil 2mbit
~# tc class add dev eth0 parent 1:1 classid 1:20 htb rate 1mbit ceil 2mbit
```

donde:

parent 1:1
: Son clases hijas de la clase `1:1`.

classid 1:*
: Son identificadores de clase.

rate 1mbit ceil 2mbit
: Son las tasas garantizada (1 Mbps) y tasa máxima permitida (2 Mbps).

Configuración resultante con el comando `tc class show`:

```
~# tc class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
```

Además de la información del comando anterior, aparecen algunos datos más que son importantes y han sido fijados a valores por defecto:

prio 0
: Es la prioridad de la clase. Por defecto es 0 (máxima prioridad)[9].

[9]Como ocurre a menudo, valores menores indican prioridades mayores.

burst 1600b cburst 1600b
: Son las ráfagas máximas (las capacidades de los cubos de tokens), expresadas aquí en bytes.

Que ambas clases tengan la misma prioridad hará que el planificador reparta la tasa excedente de forma proporcional a su `rate` a través de la clase padre. Si fueran prioridades diferentes, la clase con mayor prioridad (menor valor) utilizaría toda la tasa excedente que necesite, y solo una vez satisfecha, el sobrante estaría disponible para las otras clases.

Cada clase hija tiene garantizado 1 Mbps, pero su `ceil` le permite llegar a 2 Mbps. Cuando una de ellas no consume su tasa garantizada, esa capacidad queda disponible en la clase padre y la otra puede tomarla prestada *(borrow)* hasta alcanzar su `ceil`. Esta es la razón de ser de la jerarquía de clases.

También es muy buena idea que la clase padre tenga una tasa garantizada que sea al menos la suma de las tasas de las hijas. De otro modo se dice que hay sobresuscripción *(oversubscription)*, con lo que el planificador no puede garantizar todas las tasas garantizadas —algo similar al *overbooking* de las líneas aéreas.

Cada clase `htb` usa un mecanismo de dos cubos de tokens muy similar a trTCM con la siguiente configuración: un cubo `C` con tasa de llenado CIR=`rate` y capacidad CBS=`burst`, y un cubo `P` con tasa de llenado PIR=`ceil` y capacidad PBS=`cburst`. El resultado (los colores) se interpreta como:

- verde: puede enviar a la tasa garantizada.
- amarillo: puede pedir prestado a la clase padre (indirectamente a las clases hermanas).
- rojo: no puede enviar, tiene que esperar.

Se pueden obtener estadísticas para cada clase que nos dan una idea muy concreta del comportamiento de la política QoS. Puedes verlas con `tc -s class show`.

```
~# tc -s class show dev eth0 classid 1:10
class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 200000 ctokens: 100000
```

Las líneas que empiezan por «Sent» y «backlog» tienen el mismo significado que en el planificador, salvo que aplican solo a esta cola. Pero tenemos algunos campos nuevos que son interesantes:

borrowed: 0
: Se incrementa cada vez que, para enviar un paquete, la cola tuvo que pedir prestada tasa a otras clases.

lended: 0
: Cantidad de veces que el envío se hizo a cargo de la tasa de esta clase.

tokens: 200000
: Cantidad de tokens que quedan en el cubo `C` *(commited)*.

ctokens: 100000
: Cantidad de tokens que quedan en el cubo `P` *(peak)*.

Estas cantidades de tokens requieren una explicación. En la implementación de Linux, estos tokens son canjeables por tiempo de transmisión, en lugar de bytes, es decir, cada token representa una cantidad de tiempo que la cola puede utilizar para transmitir.

Por ejemplo, si según `burst` la capacidad del cubo `C` es 1600 bytes y su tasa garantizada es 1 Mbps (125 kBps), con el cubo lleno sería posible transmitir a esa tasa durante:

$$t = burst/rate = 1600\ bytes/125000\ bytes/s = 0{,}0128\ s = 12{,}8\ ms$$

Que con la equivalencia de 1 token=64 ns que utiliza Linux actualmente, nos da la cantidad de tokens que caben en el cubo:

$$12{,}8\ ms/64\ ns = 0{,}0128/0{,}000000064 = 200000\ tokens$$

Como el cubo `P` *(peak)* tiene una tasa de 2 Mbps, para transmitir la misma cantidad de bytes (`cburst`) solo necesita la mitad de tiempo, y por tanto la mitad de tokens.

Filtros

Los filtros conforman la lógica que decide qué clase se asocia a cada flujo, lo que a fin de cuentas constituye el clasificador del pipeline. En este pequeño ejemplo, vamos a clasificar el tráfico IP dirigido al puerto 5201 como clase `1:10` y el tráfico dirigido al puerto 5202 como clase `1:20`. El resto de tráfico se clasificará en la clase por defecto, `1:99`.

```
~# tc filter add dev eth0 protocol ip parent 1: prio 1 flower ip_proto tcp \
    dst_port 5201 flowid 1:10
~# tc filter add dev eth0 protocol ip parent 1: prio 2 flower ip_proto tcp \
    dst_port 5202 flowid 1:20
```

donde:

protocol ip
: Selecciona tráfico IPv4.

parent 1:
: Es un filtro asociado al planificador raíz.

prio 1
: Indica la prioridad (orden) en que se evalúa el filtro.

flower
: El clasificador seleccionado (`flower`) es potente y de alto nivel, permitiendo referir campos de las cabeceras de muchos protocolos. Una de sus ventajas más destacables respecto a alternativas como `u32` es que permite *hardware offload*.

ip_proto tcp
: Selecciona tráfico TCP.

dst_port 5201
: Selecciona tráfico dirigido al puerto 5201.

flowid 1:10
: Si el paquete cumple el predicado, se le clasifica como `1:10`.



Hardware offload permite instalar el clasificador en la NIC, con la consiguiente mejora de velocidad y reducción de carga de la CPU. Sin embargo, solo las NIC más modernas y de gama alta permiten esta opción.

Esquemáticamente, la configuración de las clases queda así:

```
1: (root, htb, default 99)
 +-- 1:1 (rate 2Mbps ceil 2Mbps)
 |   +-- 1:10 (rate 1Mbps ceil 2Mbps) puerto 5201
 |   +-- 1:20 (rate 1Mbps ceil 2Mbps) puerto 5202
 +-- 1:99 (rate 100Mbps) resto del tráfico
```

El comando `tc filter show` confirma esa configuración:

```
~# tc filter show dev eth0
filter parent 1: protocol ip pref 1 flower chain 0
filter parent 1: protocol ip pref 1 flower chain 0 handle 0x1 classid 1:10
  eth_type ipv4
  ip_proto tcp
  dst_port 5201
  not_in_hw
filter parent 1: protocol ip pref 2 flower chain 0
filter parent 1: protocol ip pref 2 flower chain 0 handle 0x1 classid 1:20
  eth_type ipv4
  ip_proto tcp
  dst_port 5202
  not_in_hw
```

donde:

pref *
: Es la prioridad del filtro, que determina el orden en el que se evaluarán por cada paquete. Esto es importante: los filtros más específicos deben tener mayor prioridad (menor valor) que los más generales, o serán ignorados.

chain 0
: Indica la cadena a la que pertenece el filtro. Las cadenas permiten encadenar clasificadores para construir pipelines complejos, en los que una acción puede pasar de una cadena a otra. Si no se indica otra cosa, los filtros se crean en la cadena 0.

handle 0x1
: Es el identificador del filtro, que permite referenciarlo después para modificarlo o eliminarlo. Si no se especifica, `tc` le asigna uno automáticamente.

not_in_hw
: Indica que el filtro no se ha instalado en la NIC (no hay *hardware offload*), y se ejecutará en la CPU.

Para ver los efectos de esta configuración, ejecutamos un cliente `iperf3` durante 16s contra el puerto 5201 de un servidor remoto. A los 8s y sin parar los clientes, tomamos las estadísticas de las tres clases. El script /qos/2-class-1-idle.sh implementa esta prueba; esta es la salida que genera:

```
~# tc -s class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1964834 bytes 1316 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 0b 0p requeues 0
 lended: 326 borrowed: 0 giants: 0
 tokens: -188223 ctokens: -188223

class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1964834 bytes 1316 pkt (dropped 0, overlimits 652 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 338 borrowed: 326 giants: 0
 tokens: -96473 ctokens: -188223

class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 200000 ctokens: 100000

class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
 Sent 68794 bytes 222 pkt (dropped 0, overlimits 15 requeues 0)
 backlog 0b 0p requeues 0
 lended: 209 borrowed: 0 giants: 0
 tokens: 1846 ctokens: 1846
```

En una segunda prueba (script /qos/2-class.sh) ejecutamos la misma configuración, pero con 2 clientes `iperf3` con flujos hacia los puertos 5201 y 5202 respectivamente:

```
~# tc -s class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1971868 bytes 1332 pkt (dropped 0, overlimits 1 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: -471048 ctokens: -471048

class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 985934 bytes 666 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 339 borrowed: 0 giants: 0
 tokens: -376481 ctokens: -88250
```

```
class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 985934 bytes 666 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 339 borrowed: 0 giants: 0
 tokens: -376443 ctokens: -88250

class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
 Sent 350881 bytes 572 pkt (dropped 0, overlimits 108 requeues 0)
 backlog 0b 0p requeues 0
 lended: 467 borrowed: 0 giants: 0
 tokens: 1873 ctokens: 1873
```

Y con el comando `tc qdisc show` vemos las estadísticas del planificador, con los clientes aún en ejecución:

```
~# tc -s qdisc show dev eth0
qdisc htb 1: root refcnt 2 r2q 10 default 0x99 direct_packets_stat 4 direct_qlen 1000
 Sent 2323244 bytes 1908 pkt (dropped 0, overlimits 763 requeues 0)
 backlog 12048b 4p requeues 0
```

Resultados de las clases antes y después de ejecutar el segundo cliente `iperf3`:

variable	un cliente			dos clientes		
classid	1:1	1:10	1:20	1:1	1:10	1:20
sent (pkt)	1316	1316	0	1332	666	666
overlimits	327	652	0	1	327	327
backlog (pkt)	0	2	0	0	2	2
lended	326	338	0	0	339	339
borrowed	0	326	0	0	0	0
tokens	−188 223	−96 473	200 000	−471 048	−376 481	−376 443
ctokens	−188 223	−188 223	100 000	−471 048	−88 250	−88 250

Cuadro 17.4: Estadísticas de las clases con uno o dos clientes `iperf3` contra 2 puertos distintos.

La clase `1:1` es instrumental: ningún filtro clasifica tráfico hacia ella. Su función es puramente estructural, sirve de contenedor de la tasa que sus clases hijas comparten y de la que pueden tomar prestado. Por eso sus estadísticas de envío no reflejan tráfico propio, sino la suma del de sus clases hijas.

Puedes comprobar que cuando únicamente hay tráfico en la cola `1:10`, la clase padre contabiliza su misma cantidad de paquetes enviados (1316). La cola `1:10` consumió toda la tasa disponible en el padre (2 Mbps) porque no compite con nadie. La cifra 326 indica la cantidad de paquetes que el planificador extrajo de la cola `1:10` con tokens prestados.

Cada vez que la cola `1:10` consume todos sus tokens al llegar a 1 Mbps, le pide prestado a la clase padre el permiso para enviar el paquete (326 veces `lended` en la 1:1 y `borrowed` en la `1:10`). Esto consume tokens en la clase hija y en la padre, y la cantidad consumida puede ser distinta si las tasas de esas clases son distintas (recuerda que es tiempo). Todo esto es posible solo porque la clase `1:20` no está utilizando su tasa garantizada (`rate`). Con esto, la clase `1:10` puede llegar a su tasa máxima (`ceil`) de 2 Mbps.

Resulta llamativo ver cantidades negativas de tokens. El valor negativo es el tiempo que debe transcurrir hasta que el cubo recupere tokens y la cola pueda volver a transmitir. Por tanto, encontrar estos valores negativos implica que la clase se comporta como un shaper, retrasando y no descartando, algo que queda confirmado al observar todos los `dropped` a 0.

Los signos de las variables `tokens` y `ctokens` indican además en qué modo está trabajando la clase: Si ambos son positivos, la clase está transmitiendo a su tasa garantizada o `rate` (verde). Si `tokens` es negativo y `ctokens` positivo, la clase está transmitiendo por encima de `rate`, pero por debajo de `ceil` (amarillo). Si ambos son negativos, la clase está probablemente difiriendo el tráfico (rojo), porque ha alcanzado la tasa máxima (`ceil`).

La variable `overlimits` indica la cantidad de paquetes no-verdes: es decir los que han requerido préstamos (`borrowed`) o los que tuvieron que esperar (dado que `dropped`=0). Por ejemplo, para la clase `1:10` podemos deducir:

- 1316 paquetes enviados en total.
- 326 paquetes prestados (amarillos o `borrowed`).
- 990 (1316 − 326) paquetes verdes.
- 652 por encima de la tasa garantizada (no-verdes u `overlimits`).
- 326 *overlimits* − *borrowed* = 652 − 326 **intentos** diferidos o rojos.

Lo de *intentos* es importante porque los paquetes nunca se marcan en rojo. Rojo significa que el envío se retrasa, y el mismo paquete podría ser retrasado varias veces antes de ser finalmente transmitido. Por eso, podríamos hablar de paquetes rojos si se descartaran, pero no cuando se retrasan.

Al arrancar el segundo cliente —mira las 3 últimas columnas de la Tabla 17.4— la clase `1:20` empieza a transmitir y a competir por el ancho de banda disponible. Como ahora ya no hay excedente, las dos clases tenderán a transmitir a su tasa garantizada (1 Mbps).

Policer

Para ver el efecto del policer añadimos al pipeline una tercera clase (`1:30`). También es necesario modificar la clase padre para que pueda ofrecer una tasa agregada de 3 Mbps.

```
~# tc class add dev eth0 parent 1: classid 1:1 htb rate 3mbit ceil 3mbit
~# tc class add dev eth0 parent 1:1 classid 1:30 htb rate 1mbit ceil 2mbit
```

Definimos el policer, que es un filtro con `action police` que clasifica el tráfico dirigido al puerto 5203 hacia la nueva cola 1:30.

```
~# tc filter add dev eth0 protocol ip parent 1: prio 3 flower \
  ip_proto tcp dst_port 5203 \
  action police rate 1.5mbit burst 32k conform-exceed drop \
  flowid 1:30
```

donde:

`rate 1.5mbit`
: La tasa límite (CIR): 1,5 Mbps.

`burst 32k`
: La ráfaga máxima permitida (CBS): 32 kB.

`conform-exceed drop`
: Cuando el cubo se quede sin tokens, los paquetes deben descartarse.

Este policer limita la tasa a 1,5 Mbps con ráfaga de 32 kB con un cubo simple de 2 colores. También es posible configurar un cubo trTCM con 3 colores, pero quedémonos con el de 2 colores para no complicar el ejemplo.

Hemos fijado intencionadamente la tasa objetivo del policer a 1,5 Mbps entre la tasa `rate` de la clase 1:30 (1 Mbps) y la `ceil` (2 Mbps). De este modo la nueva clase puede pedir prestado para acercarse a su `ceil` antes de llegar al techo del policer. Así podemos ver actuar a los dos mecanismos —shaper y policer, retraso y descarte— de forma complementaria.

Como en los casos anteriores, la configuración establecida incluye otros parámetros que no se han indicado al definir el filtro:

```
~# tc -s filter show dev eth0
^^Iaction order 1:  police 0x1 rate 1500Kbit burst 32Kb mtu 2Kb action drop overhead 0b
^^Iref 1 bind 1  installed 0 sec used 0 sec
^^IAction statistics:
^^ISent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
^^Ibacklog 0b 0p requeues 0
```

Otros campos con valores por defecto

`action order 1:`
: El orden de ejecución de la acción, dado que puede definir varias.

`police 0x1`
: Este filtro es un policer con identificador 0x1.

`mtu 2Kb`
: Tamaño máximo de paquete que puede pasar: 2 kB.

`action drop`
: Los paquetes que excedan (rojos) serán descartados.

overhead 0b
: Tamaño de la cabecera que se resta del tamaño del paquete para calcular el tamaño del paquete a contar.

installed 0
: El filtro se creó hace 0 segundos.

used 0
: El filtro se usó hace 0 segundos, o no se ha usado.

requeues 0
: Paquetes reencolados.

Esquemáticamente, la configuración de clases queda así:

```
1: (root, htb, default 99)
 +-- 1:1 (rate 3Mbps ceil 3Mbps)
 |   +-- 1:10 (rate 1Mbps ceil 2Mbps) puerto 5201, shaper
 |   +-- 1:20 (rate 1Mbps ceil 2Mbps) puerto 5202, shaper
 |   +-- 1:30 (rate 1Mbps ceil 2Mbps) puerto 5203, policer (1.5Mbps) + shaper
 +-- 1:99 (rate 100Mbps) resto del tráfico
```

Para probar este pipeline usamos el script /qos/policer-3-class.sh que genera flujos TCP sobre cada uno de los puertos 5201, 5202 y 5203, si bien en el caso del puerto 5203 genera 16 flujos en paralelo. Esto es necesario porque con uno o pocos flujos, el control de congestión haría su trabajo y ajustaría automáticamente la ventana en cuanto empezasen a ocurrir los RTO por los primeros descartes. Con los flujos en paralelo, el policer se ve obligado a descartar paquetes y podemos ver su efecto.

Capturamos estadísticas después de 8 segundos de ejecución, y lo primero que llama la atención es la ocupación de la cola `1:30`:

```
~# tc -s class show dev eth0 classid 1:30
class htb 1:30 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 924822 bytes 668 pkt (dropped 0, overlimits 256 requeues 0)
 backlog 96384b 32p requeues 0
 lended: 412 borrowed: 0 giants: 0
 tokens: -376475 ctokens: -88250
```

El backlog de 32 paquetes encaja con los 16 flujos TCP: los descartes del policer provocan continuas situaciones de *3dupack* y expiraciones de RTO en cada flujo, lo que mantiene su cwnd cerca del mínimo (§ 14.5). Con 16 flujos compitiendo por la misma clase, la suma de los pocos segmentos que cada uno mantiene en vuelo da como resultado ese backlog de, en promedio, 2 paquetes por flujo. La cola se ocupa porque el policer permite hasta 1,5 Mbps, pero la clase está limitada a 1 Mbps y no hay excedente que permita superarla, porque las otras clases también están emitiendo a su tope. Igual que antes, la clase se comporta como shaper, retrasando el tráfico en espera de tokens. La cola ha procesado (transmitido + retenido) un total de:

$$924\,822 + 96\,384 = 1\,021\,206 \text{ bytes}$$

Pasando a bits/s y calculando para los 8 segundos, ha aceptado datos a una tasa media de:

$$1\,021\,206 \times 8/8 \text{ s} \approx 1{,}02 \text{ Mbps}$$

Pero la tasa de transmisión efectiva fue:

$$924\,822 \times 8/8 \text{ s} \approx 0{,}92 \text{ Mbps}$$

Lo más interesante en este caso está en las estadísticas del filtro.

```
~# tc -s filter show dev eth0
^^Iaction order 1:  police 0x1 rate 1500Kbit burst 32Kb mtu 2Kb action drop overhead 0b
^^Iref 1 bind 1  installed 11 sec used 0 sec firstused 8 sec
^^IAction statistics:
^^ISent 1224516 bytes 867 pkt (dropped 71, overlimits 71 requeues 0)
^^Ibacklog 0b 0p requeues 0
```

El policer se ha encontrado 71 veces con paquetes que excedieron la ráfaga máxima (tokens agotados) y los ha descartado todos. El contador `Sent` del policer contabiliza todo lo que evalúa, aunque lo descarte. El policer ha descartado:

$$procesado - transmitido = descartado$$
$$1\,224\,516 - 1\,021\,206 = 203\,310 \text{ bytes}$$

La tasa de entrada en el filtro y la de descartes son:

$$entrada = 1\,224\,516 \times 8/8 \text{ s} \approx 1{,}22 \text{ Mbps}$$
$$descarte = 203\,310 \times 8/8 \text{ s} \approx 0{,}20 \text{ Mbps}$$

De lo que podemos concluir es que una vez se «llena» la cola, el policer descarta el tráfico excedente (25 kB/s). La tasa efectiva de la cola `1:30` no ha llegado al `rate` de 1 Mbps, pero probablemente el motivo es el arranque lento de los clientes TCP que desaprovechan tasa disponible al comienzo.

```
~# tc -s qdisc show dev eth0
qdisc htb 1: root refcnt 2 r2q 10 default 0x99 direct_packets_stat 0 direct_qlen 1000
 Sent 3171692 bytes 2484 pkt (dropped 71, overlimits 1039 requeues 0)
 backlog 108432b 36p requeues 0
```

Con este último experimento puedes ver claramente la diferencia entre la función del policer y el shaper actuando sobre el mismo tráfico. La clase `1:30` (el shaper) indica `backlog 96384b 32p`: nunca descarta, retiene para lograr una tasa media que cumpla los parámetros de la cola (y de su padre). En cambio el filtro (el policer) indica `dropped 71`, que son paquetes que superaron sus parámetros. Como descarta y no encola, el policer no añade retardo, lo que puede ser más adecuado para tráfico interactivo como voz o vídeo, que suele usar transporte UDP. Con TCP, los descartes provocarán retransmisiones y el típico perfil de tráfico en diente de sierra, algo que no es deseable para lograr flujos estables y predecibles.

17.7.3. Marcado de paquetes con netfilter

En esta sección se explica cómo usar *netfilter* (§ 11.7) para hacer marcado DSCP de los paquetes salientes de un nodo terminal, pero del mismo modo se puede usar en un router de la organización.

Por simplicidad, en este ejemplo vamos a definir solo 3 de los 12 perfiles DIFFSERV (§ 17.6).

Lo primero es crear una tabla y una cadena propias para no interferir con las que seguramente ya hay definidas en tu sistema.

```
~# nft add table ip qos_lab
~# nft add chain ip qos_lab output_chain '{ type filter hook output priority mangle;
↪  policy accept; }'
```

La primera línea crea la tabla `qos_lab` para protocolos de la familia IPV4. La segunda línea es una cadena asociada a la tabla `qos_lab` con los siguientes parámetros:

`type filter`
: Es una cadena de tipo *filter*. Permite aceptar, descartar o modificar los paquetes que la atraviesan.

`hook output`
: Está asociada al *hook output*, que significa que se aplicará a todos los paquetes generados por procesos del propio nodo. Para hacer marcado en un nodo que hace de router (reenvía) tendría que elegir el hook *forward*.

`priority mangle`
: Indica el orden en el que se evalúa la cadena. La prioridad *mangle* implica que se ejecutará antes del planificador.

`policy accept`
: Si no se indica lo contrario, la cadena aceptará los paquetes, es decir, seguirán su camino.

Por el momento hemos creado una cadena vacía que no hace nada:

```
~# nft list table ip qos_lab
table ip qos_lab {
^^Ichain output_chain {
^^I^^Itype filter hook output priority mangle; policy accept;
^^I}
}
```

Es momento de añadir la primera regla. Esto marca con el perfil EF todo el tráfico de voz/control (protocolo SIP), que usa el puerto UDP 5060, y el tráfico RTP (audio de las llamadas), que suele usar el rango de puertos UDP 10000-20000.

```
~# nft add rule ip qos_lab output_chain udp dport 5060 ip dscp set ef
~# nft add rule ip qos_lab output_chain udp dport 10000-20000 ip dscp set ef
```

La segunda clase será para datos interactivos del protocolo RDP, que utiliza el puerto TCP 3389. Este tráfico se marcará con el perfil AF31.

```
~# nft add rule ip qos_lab output_chain tcp dport 3389 ip dscp set af31
```

Y la tercera clase será para tráfico genérico *best effort*, como podría ser el generado por el programa `rsync`[10] que usa el puerto TCP 873. Lo marcaremos con el perfil CS1.

```
~# nft add rule ip qos_lab output_chain tcp dport 873 ip dscp set cs1
```

Aspecto de la configuración completa de la tabla:

```
~# nft list table ip qos_lab
table ip qos_lab {
    chain output_chain {
        type filter hook output priority mangle; policy accept;
        udp dport 5060 ip dscp set ef
        udp dport 10000-20000 ip dscp set ef
        tcp dport 3389 ip dscp set af31
        tcp dport 873 ip dscp set cs1
    }
}
```

Para clasificar este tráfico configura como raíz un planificador de tipo `prio` que es adecuado para el tráfico prioritario.

```
~# tc qdisc add dev eth0 root handle 1: prio bands 3
```

Las tres bandas especificadas se utilizan del siguiente modo:

- Banda 0 (1:1): máxima prioridad, tráfico EF (voz/control).
- Banda 1 (1:2): prioridad media, clasificado con un `htb` para los otros dos perfiles: AF31 y CS1.

[10] Un programa de copia remota masiva de ficheros

- Banda 2 (1:3): prioridad baja, resto del tráfico.

Configuración:

```
~# tc qdisc show dev eth0
qdisc prio 1: root refcnt 2 bands 3 priomap 1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
```

El mapa de prioridades `priomap` asigna a cada banda (valores 0-2) el tráfico correspondiente a cada valor de `SO_PRIORITY` (0-15), pero está pensado para los valores TOS obsoletos. Para solventarlo, vamos a crear filtros que clasifiquen según los valores DSCP que acabamos de fijar en las reglas `netfilter`.

```
# Filtro banda 0 (EF/voz) con policer
~# tc filter add dev eth0 parent 1: protocol ip prio 1 flower \
  ip_tos 0xb8/0xfc \
  action police rate 1mbit burst 10k drop \
  classid 1:1

# Filtro banda 1 (AF31/datos interactivos)
~# tc filter add dev eth0 parent 1: protocol ip prio 2 flower \
  ip_tos 0x68/0xfc \
  classid 1:2

# Filtro banda 2 (CS1/bulk)
~# tc filter add dev eth0 parent 1: protocol ip prio 3 flower \
  ip_tos 0x20/0xfc \
  classid 1:3
```

Configuración de filtros:

```
~# tc filter show dev eth0
filter parent 1: protocol ip pref 1 flower chain 0
filter parent 1: protocol ip pref 1 flower chain 0 handle 0x1 classid 1:1
  eth_type ipv4
  ip_tos 184/0xfc
  not_in_hw
filter parent 1: protocol ip pref 2 flower chain 0
filter parent 1: protocol ip pref 2 flower chain 0 handle 0x1 classid 1:2
  eth_type ipv4
  ip_tos 104/0xfc
  not_in_hw
filter parent 1: protocol ip pref 3 flower chain 0
filter parent 1: protocol ip pref 3 flower chain 0 handle 0x1 classid 1:3
  eth_type ipv4
  ip_tos 32/0xfc
  not_in_hw
```

Con lo que tenemos ahora, todo el tráfico marcado con EF se encola en la banda 1:1 y el planificador no atenderá nada más mientras haya tráfico en esa cola. Para evitar inanición en las otras clases, vamos a sustituir el primer filtro por un policer con limitación de tasa, que corresponde con un

planificador de tipo LLQ (Low Latency Queuing) y lo sustituimos por el filtro con policer:

```
~# tc filter del dev eth0 parent 1: prio 1
~# tc filter add dev eth0 parent 1: protocol ip prio 1 flower \
  ip_tos 0xb8/0xfc \
  action police rate 1mbit burst 10k drop \
  classid 1:1
```

Esto encaja directamente con un planificador LLQ de tres bandas, con la banda 0 limitada a 1 Mbps y las otras dos bandas sin limitación.

Podemos conseguir algo más flexible sustituyendo las colas FIFO de las bandas 1:2 y 1:3 por planificadores estilo CBWFQ usando `htb`.

Para ello creamos un qdisc hijo `htb` para la banda 1:2:

```
~# tc qdisc add dev eth0 parent 1:2 handle 20: htb default 1
```

Dentro de él, creamos una clase con el rate/ceil para el tráfico AF31 (datos interactivos):

```
~# tc class add dev eth0 parent 20: classid 20:1 htb rate 2mbit ceil 4mbit
```

Y para la banda 1:3 otro `htb` para el tráfico CS1 (datos genéricos):

```
~# tc qdisc add dev eth0 parent 1:3 handle 30: htb default 1
```

Ahora la clase correspondiente, con un rate/ceil menor, porque *best effort* no necesita garantías altas, solo aprovechar lo que sobre:

```
~# tc class add dev eth0 parent 30: classid 30:1 htb rate 500kbit ceil 4mbit
```

Resultado:

```
~# tc qdisc show dev eth0;
qdisc prio 1: root refcnt 2 bands 3 priomap 1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
qdisc htb 20: parent 1:2 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000
qdisc htb 30: parent 1:3 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000

~# tc class show dev eth0
class prio 1:1 parent 1:
class prio 1:2 parent 1: leaf 20:
class prio 1:3 parent 1: leaf 30:
class htb 20:1 root prio 0 rate 2Mbit ceil 4Mbit burst 1600b cburst 1600b
class htb 30:1 root prio 0 rate 500Kbit ceil 4Mbit burst 1600b cburst 1600b
```

17.8. QoS en origen

El mecanismo de encaminamiento en el propio nodo origen también puede priorizar tráfico, aunque no utilizando el marcado DSCP. Linux tiene un sistema automático llamado `ip_tos2prio`, que como su nombre indica, convierte el valor del campo `TOS` (concretamente el campo `TOS` de la RFC 1349 [56]) en una prioridad local. Desgraciadamente ese sistema utiliza la especificación obsoleta «IP precedence» de la RFC 791 [9] que no es compatible con DSCP (Figura 17.14).

En un programa actual, para tener prioridad local tendrás que hacer tanto el marcado DSCP como una selección de prioridad explícita, que tiene que ejecutarse necesariamente después para anular el efecto de la conversión automática errónea que dijimos antes. Ambas cosas se pueden hacer en la propia aplicación directamente sobre los sockets implicados. El Listado 17.7 muestra un fragmento de un servidor TCP que marca con la clase EF (la más prioritaria) usando la opción `IP_TOS` del socket y la prioridad local 6 usando la opción `SO_PRIORITY`. Estos son los niveles de prioridad local:

- `TC_PRIO_BEST_EFFORT = 0`
- `TC_PRIO_FILLER = 1`
- `TC_PRIO_BULK = 2`
- `TC_PRIO_INTERACTIVE_BULK = 4`
- `TC_PRIO_INTERACTIVE = 6`
- `TC_PRIO_CONTROL = 7`

```
DSCP_EF = 46 << 2
TC_PRIO_INTERACTIVE = 6

sock = socket.socket()
sock.bind(('', 1234))

sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, DSCP_EF)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_PRIORITY, TC_PRIO_INTERACTIVE)

sock.listen(5)

conn, addr = sock.accept()
```

LISTADO 17.7: Marcado de un flujo TCP con máxima prioridad (EF)

El servidor anterior 17.7 marca todos los segmentos de todas las conexiones que acepte ya desde su paquete SYN. Es posible marcar cada conexión independiente simplemente haciendo las mismas llamadas `setsockopt()`, pero sobre el socket conectado (`conn` en el listado). Fíjate en el desplazamiento de 2 bits (`<<2`) que se hace al valor de DSCP como corresponde a su posición

dentro del campo `TOS`. Esto deja los bits ECN a 0, pero es correcto porque el valor de esos bits es responsabilidad del núcleo.

Con UDP se puede hacer algo equivalente, es decir, marcar todos los datagramas enviados por un socket, aunque también es posible marcar cada datagrama por separado incluso con clases. Lo puedes ver en el Listado 17.8.

```
sock.sendmsg(
    [payload], [(socket.IPPROTO_IP, socket.IP_TOS, struct.pack('i', DSCP_EF))],
    0, ('example.net', 1234))
```

Listado 17.8: Marcado de un datagrama UDP individual

En cualquier caso, ten en cuenta que el efecto de la prioridad local depende del planificador configurado en la interfaz de red. El planificador `pfifo_fast` tiene tres bandas (colas de prioridad estricta) y coloca cada paquete en una u otra según el valor de `skb->priority`, traducido a número de banda mediante una tabla de 16 entradas llamada *priomap*. Otros planificadores, como `fq_codel`, ignoran totalmente la prioridad local.

Hay una vuelta de tuerca más. El valor de `SO_PRIORITY` es un entero de 32 bits, de los que la codificación de prioridad local solo emplea los 4 menos significativos (valores 0–15). El planificador `htb` interpreta ese entero como un *classid* con formato *major*:*minor*. Si el *major* es cero, algo que ocurre con todos los valores de prioridad local, es ignorado. Pero si el *major* no es cero, `htb` comprueba si hay una clase definida con ese *classid* y, si existe, encola el paquete en ella directamente, sin necesidad de definir un filtro.

Sabiendo esto, puedes escribir un programa que directamente clasifique tráfico en la cola elegida. Por ejemplo, el Listado 17.9 es un cliente UDP que coloca el flujo directamente en la cola de la clase `1:10` del ejemplo de la sección anterior:

```
CLASSID_1_10 = 0x00010010  # clase 1:10

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_PRIORITY, CLASSID_1_10)
sock.sendto(b"hola", ('example.net', 1234))
```

Listado 17.9: Marcado de un flujo UDP para la clase 1:10

Debes tener en cuenta que escribir valores de `SO_PRIORITY` mayores que 6 requiere privilegios especiales, concretamente la capacidad `CAP_NET_ADMIN`, dado que afecta de forma directa al tráfico en la red.

Y ¿qué más?

La gestión del tráfico y la QoS ofrecen formas de optimizar el uso y aprovechamiento de los recursos de la red; este capítulo ha repasado los mecanismos más importantes y cómo llevarlos a la práctica en sistemas GNU/Linux.

El tipo de QoS descrito aquí lo consigue principalmente el SO del router o nodo origen, pero estas mismas ideas se pueden aplicar en la capa de aplicación: un proxy web puede priorizar ciertas peticiones, o puede limitar la tasa de descarga o la cantidad de conexiones simultáneas que pueden realizar ciertos clientes en ciertos momentos. Estas técnicas de limitación se suelen denominar *throttling* y, a diferencia de la priorización, no buscan optimizar el uso de la red, sino salvaguardar la disponibilidad del servicio.